

28天VibeCoding学习打卡计划

写在前面

很多人第一次用 AI 编程，卡住的不是代码，而是不知道该先说什么、先做什么、做完怎么确认。

这 28 天，我们用一个小项目把这条路走一遍：先把想法说清楚，再做出能打开的页面，接着接上接口和数据库，最后把作品发到线上。

项目主题自己选。“今日热搜”只是老师贯穿使用的案例，不是统一作业。你可以做待办清单、读书笔记、课程管理，或者任何一个你愿意连续改 28 天的东西。

每天投入 45–60 分钟（以当天标注为准；其中 Day 1 含环境安装与账号注册约 60–90 分钟，Day 17 接入真实数据约 60–75 分钟，Day 7 / 8 / 12 / 15 / 20 约 90–120 分钟，Day 26 约 90 分钟）。当天只做当天的最小成果，做完就留下截图、录屏或链接。全程使用 WorkBuddy（安装、登录与注册全部在 Day 1 完成并验收，启动大会只是先演示一遍；CloudBase 从第 15 天才开始用，注册和使用方法见[附录 M](#)的指导手册，可自学），你负责说清楚要什么、核对它做得对不对。

这份手册不要求你一次学会所有技术。先让作品跑起来，再一块一块补上去。

总目录

1. 第 1 周 | 认识 Vibe Coding，启动自己的 MVP（Day 1–7）
2. 第 2 周 | 前端、交互和 Skill（Day 8–14）
3. 第 3 周 | 后端、真实数据和云端数据库（Day 15–21）
4. 第 4 周 | 测试、部署和线上验收（Day 22–28）
5. 附录

开始前先读

[返回总目录](#) · [查看打卡模板](#)

关于检测：前两周是打基础，每天的完成标准写在「完成后检查」或「实战步骤与检查」里；从第 15 天起作品要联网，每天会额外给出可以自己执行的「今天怎么检测」。这不是考核突然变严了，而是验证方式变得更具体。

课程总览

周次	主题	这一周要完成什么	本周产出
第 1 周（Day 1-7）	认识 Vibe Coding，启动自己的 MVP	装齐环境与账号 → 建仓库并首次提交 → 写 research.md、PRD.md、TECH_DESIGN.md、AGENTS.md 四份项目文档 → 用文档生成第一个能运行的 MVP	四份项目文档 + 一个能跑起来的 MVP
第 2 周（Day 8-14）	前端、交互和 Skill	生成前端主视图 → 用设计规则统一风格 → 用截图和自然语言改页面 → 加互动反馈 → 自制 Skill 完成筛选 → 多级页面与四种状态 → 轻量用户测试	一个能点、能筛、有状态的完整前端
第 3 周（Day 15-21）	后端、真实数据和云端数据库	打通云端链路并写出接口契约 → 建表与种子数据 → 读接口 → 写接口 → 数据访问层拆分 → 部署到公网并处理跨域	一个公网可访问、连真实数据库的作品
第 4 周（Day 22-28）	测试、部署和线上验收	补齐修改和删除 → 错误处理与安全边界 → 修一个真实 Bug → 做「发布前检查」Skill → 发布 v1.0 与回滚演练 → 第三方试用 → 最终验收	上线可分享的 v1.0 + 完整验收材料

每天的具体任务不在这张表里展开——打开对应那一周，从「本周 Day 目录」点进当天即可，每天的操作顺序见「开始前先读 · 1. 每天怎么做」。

1. 每天怎么做

每天按这个顺序走一遍：

1. 打开 WorkBuddy，确认工作区还是那同一个项目文件夹。
2. 复制当天「今日任务清单」整块发给 AI；当天提示词按「换成自己的项目」模板填好一并发出。
3. AI 说计划，你确认后再动手，一次只执行一步。
4. 一个板块完成它会停下汇报，你回复「进入下一板块」它才继续。
5. 全部完成后，核对 AI 自报的改动文件清单，没有陌生文件就让它提交推送。
6. 打开页面、接口或数据库，按「完成后检查」确认结果。
7. 群里回一句「每日一问」的答案 + 发一张截图（格式见 [附录 A](#)），21:00 前完成。学有余力，可以复制后面一天的任务清单，并明确说「现在开始做 Day X」。

2. 提示词五个部分

部分	用小白能懂的话说	今日热搜案例	你的项目示例

背景	我现在在做什么	多平台热搜网站	待办清单网站
目标	我希望 AI 帮我得到什么	生成热搜首页	生成任务列表
具体内容	页面或功能必须有什么	标题、卡片、平台筛选	任务名、分类、完成按钮
约束	什么不能做、要保持什么	先做最小版、不加登录	不改已有页面、不加支付
验证方式	怎样知道真的完成了	浏览器打开并显示卡片	新增任务后刷新仍存在

3. 三类任务

- 主任务：当天必须完成的最小成果。
- 有余力再做：不影响主线的加练内容。想提前做之后天数的内容，复制那天的任务清单并明确说「现在开始做 Day X」。
- 卡住时：遇到接口、网络或工具问题，先完成的替代成果。

4. 技术栈怎么选

本文提供一条可复现的示范路线：

WorkBuddy → React/Vite → CloudBase Node.js 云函数 → CloudBase PostgreSQL → CloudBase 静态网站托管

这是一条示范路线。你可以使用 Vue、纯 HTML、Python、MySQL、其他 AI 工具或其他部署平台；遇到分支时，把【[路线选择提示词](#)】发给 AI，让 AI 解释差异后再选。

5. 群内学习方式

- 每位学员每天在群里回答一个当日问题 + 发一张截图。问题在当天任务清单之后的「每日一问」里——用一道具体问题检验你是否真正掌握了当天内容：答得上来说明掌握了，答不上来就重读当天的「先讲清楚」。答题后在微信群发布打卡，早上看一眼问题，带着它去做当天的任务。
- 学员按校区分成小组，每组选一位组长。组长每天统计本校区完成情况、提醒还没交的同学，并把本组问得最多的卡点汇总一句发给老师。
- 组长每天先交自己的打卡、先提一个自己的问题，给全组打个样。组长不批改同伴作业，也不替你调代码。
- 卡住了不要憋着：在群里问，老师和其他管理者会答疑。「每日一问」答不上来、或者只能写「完成了」，就说说卡在哪一步，这也算答。腾讯会议、视频和 PPT 是可选辅助，不是完成任务的前提。
- 截图不得暴露密码、API 密钥、Cookie、完整 .env 或私人信息。

第 1 周 | 认识 Vibe Coding，启动自己的 MVP

这一周先把想法落成一个能运行的 MVP。你要能说清需求，也要能让 AI 按项目文档做出第一版。

本周 Day 目录

[Day 1](#) · [Day 2](#) · [Day 3](#) · [Day 4](#) · [Day 5](#) · [Day 6](#) · [Day 7](#)

Day 1 | 认识 Vibe Coding

今天用时：60-90 分钟（含环境安装与账号注册，这是今天的第一项验收）

[返回总目录](#) · [返回第 1 周](#) · [查看提示词五件套](#)

说明：今天的**第一项验收**是环境与账号——WorkBuddy、Git、Node.js 三项装好，GitHub 账号注册好，四项全部打勾。启动大会（开营当晚）已经演示过一遍，当时没装完很正常，今天专门留出时间装齐、装对。装完再往下做后面三项。CloudBase 属于后端环节，第 15 天才开始用，现在不需要注册（见[附录 D](#)）。

学习目标

今天要交四件事：① 装齐环境并注册账号（第一项验收）② 建好项目工作区文件夹③ 把规则文件 AGENTS.md 存进项目④ 能用自己的话说清 vibe coding 和 skill 是什么，以及为什么先立规则再动手。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 1 | 第 1 周
- 2 今天要掌握：用你自己的话说：vibe coding 是什么？为什么第一天就要先立规则？
- 3 今日板块顺序：① 装齐环境并完成账号注册（第一项验收） ② 建工作区文件夹 ③ 存规则文件 AGENTS.md ④ 说清 vibe coding 和 skill
- 4 主任务：装齐环境（WorkBuddy / Git / Node.js）+ 注册 GitHub 账号 + 建好工作区文件夹，并把项目规则文件 AGENTS.md 存进去
- 5 今日不做：写代码、建仓库（Day 2 才做）
- 6 完成标准：环境与账号四项全部打勾（WorkBuddy、Git、Node.js、GitHub 账号）；项目文件夹里有 AGENTS.md；能用自己的话解释 vibe coding 和 skill
- 7 今天交什么截图：文件管理器打开你的项目工作区文件夹——图里要有完整的文件夹路径，并且能看到文件夹里的 AGENTS.md
- 8 收尾指令：全部完成后告诉我完成情况并保留截图。今天还没有代码仓库，不需要提交代码
- 9 余力加练：让 AI 用零基础能懂的说法解释什么是前端、后端、数据库
- 10 卡住降级：某一项装不上就先跳过并记为待办，明天上课前问老师，不要卡在一项上；找不到工作区文件夹就先在桌面新建一个，用你的项目名命名；存规则文件失败就手动新建 AGENTS.md，再把模板整段粘进去

先讲清楚

vibe coding 的意思是「说人话就能做软件」：你用日常语言把想法告诉 AI，AI 帮你把想法变成能运行的程序。它不要求你先学会编程，但要求你把话说清楚、做完亲自检查。

skill 是给 AI 的一套工作说明，就像厨师做菜步骤——先备什么料、按什么顺序做、做成什么样算完成。把步骤写清楚，AI 每次都能按同样的标准干活，这个概念第 2 周会亲手用到。

我们全程使用的工具是 WorkBuddy，一个桌面 AI 编程助手：它能读写你电脑上的项目文件夹、运行命令、帮你把代码部署上线。从明天开始，仓库、代码、部署都由它代劳，你负责说清楚要什么、核对它做得对不对。

实战步骤

1. 在一个你记得住的位置新建一个文件夹，这个文件夹就是你的工作区。记住它，以后 28 天每天 WorkBuddy 都打开这同一个文件夹。
2. 把下面的 AGENTS.md 模板整段复制发给 AI，说「把这段规则存成我项目里的 AGENTS.md 文件」。AI 存好后打开项目文件夹，能看到 AGENTS.md 这个文件 = 成功。

AGENTS.md 模板（整段复制发给 AI）

代码块

- 1 # 项目规则（AGENTS.md）
- 2
- 3 我每天会把「今日任务清单」整块发给你，那是我今天要做的全部内容。请严格按下面的规则配合我，不要做清单以外的事。
- 4

一、任务边界（最重要）

1. 收到我的任务清单后，**这份清单就是今天唯一的任务范围**，只做清单里写的事。
2. 不做清单以外的工作：不建多余的文件、不写清单没提的功能、不顺手优化已有代码。你觉得可以改得更好，先跟我说一句，等我点头。
3. 不提前做以后的天数。只有我把某天的清单发给你、并明确说「现在开始做 Day X」，才允许动那天。
4. 拿不准是否在范围内，先问我，不要猜。

二、交互式推进

1. 收到清单后，先只给我今天的执行计划：按什么顺序做、每步怎么做、哪些要我自己动手。**写完停下，等我确认。***
2. 确认后**一次只做一个步骤**，绝不连着往下做。
3. 每步做完立刻停下，告诉我三件事：做了什么、改了哪些文件、怎么验证（附我能亲眼确认的结果）。
4. 然后**停下等我回复「进入下一板块」**。我没说这四个字，你就不要继续。
5. 「进入下一板块」只对当前这一步有效，不代表可以一路做到收尾。

三、人机分工

1. 需要我人工操作的（注册账号、点按钮、装软件、截图等）：不要替我做，也不要假装已完成。只告诉我：这步在干什么、一二三步怎么做、做完我该看到什么、容易踩什么坑，然后停下等我。
2. 需要你生成的内容（写文档、建文件、写代码）：**等我说了再做**，不要在我刚说「开始吧」时就把整天的东西生成出来。
3. 需要拍板的地方（选方案、起名字、定风格），把选项和各自代价列给我选，不要自己决定。
4. 验证结果要我能亲眼确认（页面能打开、文件能看到、命令有输出），不接受「已经完成了」这种口头结论。

四、讲清楚环节（我是来学的）

1. 每个步骤开始前，先告诉我：它在整个作品里是什么位置、为什么现在做、做完会变成什么样。
2. 每步用「做了什么 → 结果 → 为什么」讲明白，不要只给结果。
3. 我不懂的概念（命令、报错、术语、配置），一两句话解释清楚，不要默认我会。
4. 报错时先告诉我错在哪、可能什么原因、打算怎么修，我同意后再动手。不要输出教程式长文。

五、Git 与提交

1. 我授权你代为执行 git 提交和推送，但**提交前必须先列出**本次会改动哪些文件、每个文件属于哪天的任务，等我确认后再提交。
2. commit 信息格式：`Day X | 一句话说明`。
3. 密钥、`.env`、数据库连接串永远不进代码、不进提交。
4. 一天的任务全部做完、我核对完文件清单之后才提交，中途不要边做边提交。

六、当日收尾

- 43
1. 我说「今天做完了」之后，请你逐条对照清单里的「完成标准」，输出一张表：完成标准 / 状态（已完成·部分完成·未完成）/ 证据（文件、页面、命令输出）。
- 44
2. 核对必须诚实，没做完、做了一半、跳过没做的，直接标出来并说明原因，不要为了凑「全部完成」而模糊处理。
- 45
3. 有未完成项时，告诉我还差什么、接不接着做，**由我决定**，不要自己往下做。
- 46
4. 全部完成时，按清单的「收尾指令」整理当天成果，提醒我该交哪张截图、该在群里回什么。
- 47
-
- 48
- ## 七、出问题的时候
- 49
-
- 50
1. 我要求撤销时，用安全回退（`git revert`，保留历史），禁止 `git reset --hard` 和强制推送。
- 51
2. 回退前先和我确认恢复到哪个提交，回退后列出当前状态让我核对。
- 52
3. 同一个操作反复失败时停下来报告，说清楚试了什么、报什么错、你判断的原因，不要循环重试。
- 53
-

完成后检查

- **第一项验收：**环境与账号——WorkBuddy 能打开并读到项目文件夹；Git 装好；Node.js 装好；GitHub 账号已注册并能登录。四项打勾才算过。
- 项目文件夹里能看到 AGENTS.md。
- 能用自己的话说清 vibe coding 和 skill 各是什么；答不上来就重读「先讲清楚」。

每日一问

用你自己的话说：vibe coding 是什么？为什么第一天就要先立规则？

Day 2 | 战前部署：准备工具和 GitHub

今天用时：45-60 分钟

[返回总目录](#) · [返回第 1 周](#) · [查看求助模板](#)

学习目标

让 AI 完成仓库创建、生成一个 index.html 占位页并完成第一次提交，你能在 GitHub 网页上看到它们。

今日任务清单（复制发给 AI）

代码块

- 1
- 【今日任务清单】Day 2 | 第 1 周
- 2
- 今天要掌握：第一次把项目存进 GitHub 时，你最担心哪一步出错？为什么？

- 3 今日板块顺序：① Git 与 GitHub 概念 ② 仓库创建与首次提交（AI 代办）
- 4 主任务：完成 GitHub 仓库和第一次提交（AI 生成 index.html 占位页）
- 5 今日不做：网站功能开发（Day 7 才做 MVP）；背 Git 命令（会指挥 AI 就行）
- 6 完成标准：GitHub 网页能看到仓库和 index.html；忽略文件生效，.env 不会被上传（.env 是以后存数据库配置的本地文件，第 23 天才会用到，今天只要确保忽略规则里包含它）
- 7 今天交什么截图：GitHub 网页上你的仓库首页——图里要有仓库名、index.html 文件名，以及 commit 信息以「Day 2 |」开头的提交记录
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 2 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：让 AI 解释如何在本地回退到刚才的版本
- 10 卡住降级：GitHub 注册受阻，先完成本地 Git 初始化并保存终端截图，Day 7 之前补齐即可，期间只用本地 Git

先讲清楚

Git 是版本记录工具，GitHub 是云端代码仓库。今天不要求记住所有命令，只要让 AI 根据你的电脑环境生成清单并完成第一次保存。

开始前准备

Day 1 已经装好了 WorkBuddy、Git 和 Node.js，GitHub 账号也注册好了。今天开始前请确认三件事：WorkBuddy 能打开你的项目文件夹；终端里输入 `git --version` 和 `node -v` 都能显示出版本号；你知道哪些文件不该上传（`.env`、密钥、依赖目录）。任何一项没完成，先回到 Day 1 的环境与账号部分补齐。另外明确一点：本课程不需要额外安装代码编辑器，在 WorkBuddy 里改就行；如果你想装一个，用 VS Code。

实战案例：今日热搜

代码块

- 1 我是 Git 初学者，当前项目是“今日热搜”网站。
- 2 请先检查当前文件夹、操作系统和已经安装的工具，再给我一份把项目保存到 GitHub 的逐步清单。
- 3 每一步说明：在哪个文件夹执行、命令做什么、成功时看到什么。
- 4 请创建合适的忽略文件，禁止上传 `.env`、密码、API 密钥、依赖目录和本地数据库文件。
- 5 如果我的环境缺少工具，请先列出缺什么，不要直接执行高风险安装。

提示词怎么改

- 背景：项目名称、你的操作系统和 Git 熟悉程度。
- 目标：建立远程仓库并保存当前版本。
- 约束：敏感文件不能上传；命令要逐步解释。
- 验证：GitHub 页面能看到代码，且忽略文件生效。

把“今日热搜”换成你的项目，把“Git 初学者”换成自己的情况；如果你没有 GitHub 账号，要在提示词中明确说明。

换成自己的项目

代码块

- 1 我是【操作系统】上的 Git 初学者，当前项目叫【项目名称】，项目文件夹是【文件夹位置】。
- 2 请先检查我的环境，再给我创建 GitHub 仓库并保存第一次版本的逐步清单。
- 3 每一步说明位置、命令作用和成功现象；请保护【我不能公开的文件或变量】。
- 4 验证方式：我会在 GitHub 看到【仓库/文件】并确认【敏感文件没有出现】。

实战步骤

- 1. 让 AI 先检查环境，不要直接复制不适合你电脑的命令。Day 1 已装好这些工具，这一步主要是让 AI 确认版本和路径可用。
- 2. 创建仓库和忽略文件。**仓库建在班级的 GitHub 组织下，建成公开仓库。**加入组织的流程：① 把你的 GitHub 用户名发给老师/组长统一统计；② 收到组织邀请后，打开邀请邮件或 GitHub 通知页点 Accept；③ 新建仓库时 Owner 选班级组织，Visibility 选 Public。建完把仓库链接发到群里，同学之间可以互相看。**备选：**如果还没收到邀请或加入失败，先建个人公开仓库发到群里，等加入组织后再把仓库转移过去（转移步骤问 AI 或老师），不阻塞当天任务。
- 3. 按清单执行一次保存，并在 GitHub 网页刷新确认。

完成后检查

检查仓库代码、`.gitignore` 忽略文件和提交记录；用搜索确认仓库里没有 `.env`、数据库文件或密钥。

每日一问

第一次把项目存进 GitHub 时，你最担心哪一步出错？为什么？

Day 3 | 理解项目开发流程，完成需求研究

今天用时：45-60 分钟

[返回总目录](#) · [返回第 1 周](#) · [查看术语表](#)

学习目标

学会在写代码前先研究需求，生成一份 research.md，知道类似产品做了什么、你的项目先做什么。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 3 | 第 1 周
- 2 今天要掌握：你研究的哪个产品改变了你原本的想法？改变在哪？
- 3 今日板块顺序：① Vibe Coding 五步工作流 ② 需求研究 ③ 产出 research.md
- 4 主任务：完成自己的 research.md
- 5 今日不做：写 PRD (Day 4)；写任何代码
- 6 完成标准：research.md 已保存进仓库；包含 2-3 个类似产品比较；有「本期不做」清单
- 7 今天交什么截图：打开的 research.md 内容——图里要有 2-3 个类似产品的比较，以及「本期不做」清单
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 3 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：请一位同学只看研究文档，复述你的项目要解决什么问题
- 10 卡住降级：网络不可用时，用你知道的两个产品做经验比较，并标注「待核实」

先讲清楚：Vibe Coding 五步工作流

1. 研究需求；2. 写 PRD；3. 技术设计；4. 保存项目规则；5. 小步开发和迭代。先想清楚再动手，能减少返工。

实战案例：今日热搜

代码块

- 1 我准备制作一个“今日热搜”网站，请先不要写代码。
- 2 请研究 2-3 个类似产品，比较它们的目标用户、核心页面、主要数据、明显优点和不足。
- 3 再根据研究结果提出一个适合零基础在 28 天内完成的最小版本，明确列出“本期不做”的功能。
- 4 把结果保存为 research.md，并在最后列出你使用了哪些公开参考资料。

提示词怎么改

“不要写代码”是阶段约束；“2-3 个类似产品”是研究范围；“目标用户、页面、数据、优缺点”是比较维度；“28 天内完成”是时间边界；research.md 是交付物。把项目名称、类似产品、研究维度和时间换成自己的内容。

换成自己的项目

代码块

- 1 我准备制作【项目名称】，目标用户是【用户】。
- 2 请先不要写代码，研究【2-3 个类似产品/网站】。
- 3 比较【页面结构】、【主要数据】、【用户能完成的动作】、优点和不足。
- 4 最后提出我在【时间范围】内能完成的最小版本，并列出不做的功能。

实战步骤

1. 先让 AI 说明资料来源和不确定点。
2. 阅读生成的 research.md，删掉与你的项目无关的内容。
3. 补写一段“我最终决定先做什么，为什么”。

完成后检查

文件存在、至少有两个参考对象、有明确 MVP 和不做清单；没有把搜索结果直接当成事实而不标注来源。

每日一问

你研究的哪个产品改变了你原本的想法？改变在哪？

Day 4 | 撰写产品需求文档（PRD）

今天用时：45-60 分钟

[返回总目录](#) · [返回第 1 周](#) · [查看提示词五件套](#)

学习目标

把“我想做一个网站”变成 AI 能执行的产品需求文档 PRD.md。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 4 | 第 1 周
- 2 今天要掌握：你在 PRD 里砍掉了什么功能？为什么砍它？
- 3 今日板块顺序：① PRD 结构讲解 ② 撰写 PRD ③ AI 自检
- 4 主任务：完成 PRD.md 并让 AI 按验收标准自检一次
- 5 今日不做：技术选型（Day 5）；写代码
- 6 完成标准：PRD.md 已入库；含验收标准；AI 自检挑不出「必须修改」级问题
- 7 今天交什么截图：打开的 PRD.md 内容——图里要有验收标准那一段，并能看出是哪个项目的 PRD
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 4 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 把一项功能写成用户故事
- 10 卡住降级：只完成首页、详情和一项写入功能的 PRD，其他内容列后续

先讲清楚

PRD 不是代码，而是产品总纲。至少要说清楚：给谁用、解决什么问题、MVP 有哪些功能、每个页面有什么、哪些功能以后再做。

实战案例：今日热搜

代码块

- 1 请根据 research.md，为“今日热搜”写一份适合零基础实现的 PRD.md。
- 2 必须包含：产品概述、目标用户、MVP 功能、后续功能、页面与交互要求、数据字段、异常状态、验收标准。
- 3 MVP 先包含首页热搜列表、详情查看、至少一个收藏或备注动作；登录、复杂推荐和付费功能列为后续。
- 4 每个功能都写出用户动作和可观察结果，不要直接写代码。

提示词怎么改

“根据 research.md”让 AI 使用已有上下文；“MVP/后续”控制范围；“用户动作和可观察结果”避免只写抽象口号；“不要写代码”保持今天的任务边界。把项目、核心动作、页面和不做清单替换成自己的内容。

换成自己的项目

代码块

- 1 请根据【我的研究文档路径】，为【项目名称】写 PRD.md。
- 2 目标用户是【用户】，要解决【问题】。
- 3 MVP 必须包含【功能 1】【功能 2】【功能 3】，每个功能写清用户动作和可观察结果。
- 4 页面需要有【页面/视图列表】，数据字段包括【字段】。
- 5 把【暂时不做的功能】列为后续，不要写代码。
- 6 最后给出可逐条检查的验收标准。

实战步骤

1. 先让 AI 阅读研究文档，再生成 PRD。
2. 检查每个 MVP 功能是否能在一天内被拆成小任务。
3. 删除“以后再说但没有边界”的空泛表述。

完成后检查

随机挑一个功能，能回答“谁、在什么页面、做什么、看到什么”；PRD 有 MVP 和后续分界；至少有三个页面或视图的初步规划。

每日一问

你在 PRD 里砍掉了什么功能？为什么砍它？

Day 5 | 技术与架构

今天用时：45-60 分钟

[返回总目录](#) · [返回第 1 周](#) · [查看数据库比较](#)

今天不自己写技术文档。把 PRD 交给 AI，让它参考示范路线给出技术建议，你懂它的理由后确认。

学习目标

让 AI 根据 PRD 提出一条能跑通的技术路线，并保存为 TECH_DESIGN.md。技术栈可以换，但选择必须有理由。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 5 | 第 1 周
- 2 今天要掌握：用一句话说清你的数据从哪来、到哪去。
- 3 今日板块顺序：① 前后端与数据库分工 ② 技术路线生成 ③ 数据流图
- 4 主任务：完成 TECH_DESIGN.md 和一张简单数据流图
- 5 今日不做：写代码；回头大改 PRD
- 6 完成标准：TECH_DESIGN.md 已入库；技术路线有理由；数据流图能说清数据从哪来、到哪去
- 7 今天交什么截图：打开的 TECH_DESIGN.md——图里要有数据流图，以及技术路线那一行
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 5 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：让 AI 生成「方案变化时哪些文件会受影响」的表格；了解什么是技术栈——问 AI 开发网页或 app 会用到哪些技术、各有什么用、优劣势是什么
- 10 卡住降级：采用示范路线，不做多方案比较，但要写出为什么暂时选它

先讲清楚

前端负责展示和交互，后端负责业务规则和数据接口，数据库负责长期保存。架构图不是考试图，而是帮助你知悉数据从哪里来、到哪里去。

实战案例：今日热搜

代码块

- 1 请根据 PRD.md 为“今日热搜”生成 TECH_DESIGN.md。
- 2 先比较 2-3 套适合零基础的前端、后端、数据库和部署方案，再推荐一套默认路线并说明取舍。

- 3 示范路线可使用 React/Vite、CloudBase Node.js 云函数、CloudBase PostgreSQL 和 CloudBase 静态网站托管。
- 4 文档必须包含项目结构、数据模型、API 列表、前后端数据流、错误处理、环境变量和迁移注意事项。
- 5 不要凭空创建文件；如果信息不足，先列问题再给临时假设。

提示词怎么改

比较方案是为了理解取舍，不是要求你全部实现；默认路线是“照着跑”的例子；项目结构、数据模型和 API 是后续每天的共同语言；环境变量和迁移是在部署时避免数据丢失的约束。替换项目对象、数据字段和你能接受的复杂度。

换成自己的项目

代码块

- 1 请根据【PRD 文件】为【项目名称】生成 TECH_DESIGN.md。
- 2 请比较【方案 A】【方案 B】的学习成本、线上持久化、费用和排错难度，推荐一套并说明原因。
- 3 文档必须包含：项目结构、【数据对象】及字段、API 列表、数据流、错误处理、环境变量、部署和迁移注意事项。
- 4 我的限制是【电脑/时间/账号限制】；信息不足时先列问题，不要直接编造文件。

实战步骤

1. 阅读 AI 的方案比较，确认自己看得懂推荐理由。
2. 检查项目结构是否能对应 PRD 的页面和功能。
3. 保存 TECH_DESIGN.md，标注尚未决定的地方。

完成后检查

能指着某张表说出：它显示在哪个页面、由哪个接口读写；每个 MVP 写入/读取动作都有 API 或本地替代方案；线上数据库持久化有说明。

每日一问

用一句话说清你的数据从哪来、到哪去。

Day 6 | 创建自己的项目规则文件（AGENTS.md）

今天用时：45–60 分钟

[返回总目录](#) · [返回第 1 周](#) · [查看 Skill 说明](#)

学习目标

把“希望 AI 遵守的项目规则”保存下来，让 AI 每次工作前都先读规则、先计划、再小步执行。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 6 | 第 1 周
- 2 今天要掌握：你给自己项目加了哪条新规则？它防的是哪种情况？
- 3 今日板块顺序：① 检查 Day 1 的规则文件 ② 追加自己的规则 ③ 让 AI 复述确认
- 4 主任务：检查并完善项目规则文件（AGENTS.md）
- 5 今日不做：重写整个规则文件（只允许追加）；写代码
- 6 完成标准：AGENTS.md 在项目里，Day 1 写入的规则仍在；重点核对「当天只做当天任务，学员没说『进入下一板块』就不越界」这条还在；追加了自己至少 1 条规则；AI 能复述出规则内容
- 7 今天交什么截图：打开项目里的 AGENTS.md——图里要有你自己追加的那条规则，并且原有的「当天只做当天任务」那条仍在
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 6 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：把其中一条规则改写成队友也能用的检查项
- 10 卡住降级：规则文件丢失就重新粘贴 Day 1 模板，再追加自己的规则

先讲清楚

AGENTS.md 是 WorkBuddy 每次开工都会先读取的项目规则文件。你在第 1 天已经把它存进了项目，今天要做的是确认它真的生效，再补上你自己的规则。

追加规则（在 Day 1 的 AGENTS.md 末尾改）

代码块

- 1 在 AGENTS.md 文件末尾追加，不要改动已有内容。追加示例：
- 2
- 3 ## 我的个人规则
- 4 - 回答和注释全部使用中文。
- 5 - 我的代码放在 my-app 文件夹里，其他文件夹不要动。
- 6 - 每次改完代码，提醒我刷新浏览器看效果。

提示词怎么改

今天不需要背提示词——规则写在文件里，AI 每次自动读。你要做的只有两件事：检查文件还在不在，往末尾追加自己的规则。

换成自己的项目

代码块

- 1 请读取我项目里的 AGENTS.md。
- 2 我的项目是【项目名称】，我会用它来【一句话说明】。
- 3 请基于这个项目，建议 3 条适合追加到 AGENTS.md 的个人规则（比如文件存放习惯、语言偏好、验证习惯），每条说明理由。
- 4 我确认后再让你追加进去。

实战步骤

1. 打开 AGENTS.md 确认内容还在，然后让 AI 复述一遍文件里的规则，验证它真的读到了。
2. 发一个很小的测试任务，观察 AI 是否先说明计划并给出验证证据。

完成后检查

规则文件存在；测试任务中 AI 没有跳过计划；完成后说明修改文件和验证结果；没有把密钥或私人路径写入规则。

每日一问

你给自己项目加了哪条新规则？它防的是哪种情况？

Day 7 | 用项目文档生成第一个可运行版本

今天用时：90-120 分钟

今天第一次让项目真正跑起来，卡住是正常的，不是你太慢。每一步都让 AI 给出「成功时应该看到什么」，看不到就把报错原文发给它。

[返回总目录](#) · [返回第 1 周](#) · [进入第 2 周](#)

学习目标

用研究文档、PRD、技术设计和项目规则生成自己的 MVP。今天先让它能运行，并保留后续修改的空间。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 7 | 第 1 周
- 2 今天要掌握：第一版跑起来的那一刻，和你原本想象的差在哪？
- 3 今日板块顺序：① 读四份项目文档 ② 分步生成 MVP ③ 运行与验证
- 4 主任务：完成一个能运行的 MVP，并保存运行说明

- 5 今日不做：登录、支付、复杂缓存等未经确认的功能；一次生成大量代码
- 6 完成标准：页面能在本地启动打开；运行命令已存档；改动已提交
- 7 今天交什么截图：浏览器里打开的页面——图里要有 localhost 开头的地址栏，页面内容能认出是你的项目
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 7 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 生成一张当前项目结构图
- 10 卡住降级：只做页面骨架和一份可执行的后续计划，不追求完整功能

先讲清楚

MVP 是最小可行产品：不是把所有想法都做完，而是用最少功能验证核心价值。遇到 AI 一次生成太多代码时，要把任务拆成“先骨架、再一个功能、再验证”。

实战案例：今日热搜（例子）

代码块

- 1 请先阅读 research.md、PRD.md、TECH_DESIGN.md 和 AGENTS.md。
- 2 我要启动“今日热搜”的 MVP。请先给出不超过 5 步的实施计划，第一步只生成能启动的页面骨架。
- 3 每一步完成后说明修改了哪些文件、如何运行和如何验证；不要同时添加登录、支付、复杂缓存或未经确认的功能。
- 4 如果现有文件与文档冲突，先列出冲突并等待我选择。

提示词怎么改

“先阅读”保证 AI 使用上下文；“不超过 5 步”控制范围；“第一步只生成骨架”降低失败成本；“冲突先停”防止 AI 擅自改需求。替换项目名、文档名、第一步成果和禁止功能。

换成自己的项目

代码块

- 1 请先阅读【我的研究/PRD/技术设计/规则文件】。
- 2 我要启动【项目名称】的 MVP。请给出不超过【步数】步的计划，第一步只完成【最小可运行成果】。
- 3 每一步说明修改文件、运行方法和验证方法；暂时不要做【禁止功能】。
- 4 如果文档和代码冲突，请先列出冲突，不要自行决定。

实战步骤

1. 先让 AI 只做计划，确认计划没有超出 PRD。
2. 执行第一步并运行；通过后再执行下一步。

3. 保存启动命令、项目结构和当前截图。

完成后检查

前端的占位页面能在浏览器里打开；首页、列表等已实现的区块能正常显示；哪一项还没实现，明确写下来是哪一项，作为 Day 8 的起点。

每日一问

第一版跑起来的那一刻，和你原本想象的差在哪？

今天同时是周验证日（第 1 周）：除了下面这一问和一张截图，还要按 [附录 A](#) 的第三张卡，多交一份完整材料。

第 2 周 | 前端、交互和 Skill

注意：本周把前端补全，让它从一张静态页面变成能操作的界面——有切换、有反馈、有状态。提前说明一件事：这个页面在第 3 周会被部署上线，界面上还会再加一块数据检查区，到时候统一叫它「检查台」，不用觉得是换了个东西。

[返回总目录](#) · [本周 Day 目录](#)

这一周做完：你有至少 3 个页面或独立视图，至少 1 个有明确反馈的交互，并完成一次 Skill 的创建与调用。

本周 Day 目录

[Day 8](#) · [Day 9](#) · [Day 10](#) · [Day 11](#) · [Day 12](#) · [Day 13](#) · [Day 14](#)

Day 8 | 写清需求，一次生成网站前端

今天用时：90–120 分钟

[返回总目录](#) · [返回第 2 周](#) · [查看提示词五件套](#)

学习目标

根据 PRD 生成前端首页或主视图，理解“页面目标、内容结构、视觉约束、响应式和验证”如何写进提示词。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 8 | 第 2 周
- 2 今天要掌握：四种页面状态里，你觉得哪种最容易被忽略？为什么？
- 3 今日板块顺序：① 一句话生成页面 ② 本地运行 ③ mock 数据渲染
- 4 主任务：生成并运行项目主视图（mock 数据版）
- 5 今日不做：接真实 API（第 3 周才做）；加登录
- 6 完成标准：页面本地能打开；列表和卡片用本地假数据渲染；改动已提交
- 7 今天交什么截图：浏览器里打开的页面主视图——图里要有地址栏，以及已经渲染出来的列表或卡片
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 8 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 生成一个可复用卡片或列表组件
- 10 卡住降级：只完成静态数据版，保留加载、空和错误状态

先讲清楚

前端是用户看到和操作的部分。页面不是一张海报，而是有内容层级、按钮状态、空状态和移动端布局的可用界面。技术栈可以是 React、Vue 或纯 HTML。

实战案例：今日热搜

代码块

- 1 请根据 PRD.md 生成“今日热搜”首页前端。
- 2 页面目标：用户打开后能快速浏览多个平台的热门话题。
- 3 内容结构：顶部标题和更新时间、微博/抖音/B站三个平台并列展示（每个平台一栏，栏内显示该平台当天热搜的若干条，序号在前、热度在右）、热搜卡片列表、加载中、成功、空、错误四种状态。
- 4 视觉要求：信息层级清楚，卡片可扫描，桌面多栏并排、手机宽度下改为纵向堆叠且不横向溢出；不要添加 PRD 没有的登录和支付。
- 5 请先说明页面结构和需要修改的文件，再生成代码；完成后说明桌面宽度和手机宽度的验证步骤。

提示词怎么改

目标说明“用户为什么打开”；内容结构说明“页面必须有什么”；视觉要求说明“看起来和用起来要怎样”；限制防止 AI 自作主张；验证要求把审美变成可检查动作。把热搜、卡片、筛选和状态换成你的数据对象。

换成自己的项目

代码块

- 1 请根据【PRD 路径】生成【项目名称】的【首页/主视图】。
- 2 页面目标是让【目标用户】完成【核心动作】。
- 3 内容结构必须包含【区块 1】【区块 2】【区块 3】，还要有【加载中/成功/空/错误四种状态】。
- 4 视觉要求是【风格、色彩、信息密度】，桌面和手机宽度都要能使用。

实战步骤

1. 先让 AI 输出页面结构，不满意先改结构再生成代码。
2. 运行页面，分别用桌面宽度和手机宽度查看。
3. 用一条追问修正一个明显问题。

完成后检查

主视图能运行；用户一眼能找到核心动作；加载中、成功、空、错误四种状态都能看到（成功态就是正常显示，不需要额外造一个特殊情况）；缩窄浏览器后没有横向溢出。

每日一问

四种页面状态里，你觉得哪种最容易被忽略？为什么？

Day 9 | 用设计规则统一页面风格

今天用时：45-60 分钟

[返回总目录](#) · [返回第 2 周](#) · [查看 Skill 说明](#)

学习目标

用一份设计规则或现成 Skill 统一页面的颜色、字体、间距、组件和响应式行为，理解“好看”必须变成可以执行的规则。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 9 | 第 2 周
- 2 今天要掌握：今天改的几处里，哪一处对「像不像一个正经产品」影响最大？
- 3 今日板块顺序：① 了解设计规则 ② 用规则做前端审查 ③ 修复
- 4 主任务：用设计规则完成一次前端审查和修复
- 5 今日不做：大改版面结构；新增页面
- 6 完成标准：至少修掉 3 处设计问题（文字对比、间距、对齐等）；修前后截图各一张
- 7 今天交什么截图：页面修复前、修复后两张——图里要有地址栏，能看出同一位置的对比度、间距或对齐发生了变化
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 9 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：让 AI 把一条视觉规则写成可复用组件约束

先讲清楚

Skill 是可重复使用的工作方法，不是魔法滤镜。它应该告诉 AI 修改前检查什么、修改时遵守什么、修改后怎样验证。今天可以使用老师提供的 Skill，也要看懂其中的规则。

今天用的 Skill 可以用老师提供的，也可以自己上网找一个喜欢的。不管用哪一个，都要能说出它检查了什么、遵守了什么、怎么验证——这是第 3 周和第 4 周亲手做 Skill 的基础。

实战案例：今日热搜

代码块

- 1 请读取当前“今日热搜”页面和现有设计规则。
- 2 用设计 Skill 检查并改善：信息层级、字体大小、颜色对比、卡片间距、按钮状态、手机宽度布局。
- 3 不要添加渐变背景、随机装饰或与产品无关的组件；先列出最多 5 个具体问题（不足如实说明），再逐个修复。
- 4 每次修改后说明文件和验证方法，保持现有功能不变。

提示词怎么改

“读取现有页面”避免重新设计；五个检查维度可替换成你的界面问题；“不要添加”是审美边界；“功能不变”保护业务行为；“逐个修复”方便回退。

换成自己的项目

代码块

- 1 请读取【我的页面】和【我的设计规则/Skill】。
- 2 检查并改善【信息层级】【字体】【颜色对比】【间距】【按钮状态】【移动端】中的问题。
- 3 先列出最多 5 个具体问题（不足如实说明），再逐个修复；不要添加【不需要的视觉元素】。
- 4 保持【必须不变的功能】，每次修改后说明文件和验证方法。

实战步骤

1. 先让 AI 只做审查，截图记录问题。
2. 一次修一个问题，修完刷新页面。
3. 在手机宽度重新检查文字、按钮和卡片。

完成后检查

至少有三处可指出的改善；文字和按钮没有溢出；交互没有因为美化而失效；每处改善都有前后证据。

每日一问

今天改的几处里，哪一处对「像不像一个正经产品」影响最大？

Day 10 | 用截图和自然语言修改页面

今天用时：45-60 分钟

[返回总目录](#) · [返回第 2 周](#)

学习目标

学会描述“哪里不对、希望怎样、什么不能变”，让 AI 根据截图或文字完成一次精确修改。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 10 | 第 2 周
- 2 今天要掌握：你怎么描述那个问题，让 AI 一次就改对了（或改错了）？
- 3 今日板块顺序：① 截图定位问题 ② 自然语言描述修改 ③ 验证
- 4 主任务：使用截图和自然语言修复一个真实前端问题
- 5 今日不做：一次让 AI 改一堆问题（一次只改一个）
- 6 完成标准：问题修复且页面正常；修复前后截图；改动已提交
- 7 今天交什么截图：页面修复前、修复后两张——图里要有地址栏，能看出原来的那个问题已经消失
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 10 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 给出修复前后的差异摘要
- 10 卡住降级：没有明显问题，就主动把一个按钮或表单的反馈描述得更清楚并验证

先讲清楚

“不好看”对 AI 太模糊。有效反馈至少包含位置、现象、目标、保持不变的内容和验证方式。截图用于指出位置，自然语言用于说明意图。

注意：每个人的页面风格不同，截图内容可以随你选；发到群里之前记得遮掉私人信息（账号、手机号、本地文件路径等）。

实战案例：今日热搜

代码块

- 1 我在“今日热搜”首页发现：手机宽度下平台筛选按钮换行后遮住了更新时间。
- 2 请先定位负责布局的文件，解释可能原因；只调整筛选区的布局和间距，不改变数据、接口和其他卡片。
- 3 完成后告诉我如何在 360px 宽度下验证没有遮挡。

提示词怎么改

现象要可观察；位置要具体；“只调整”限制影响范围；“不改变数据和接口”保护功能；360px 是可执行的验证条件。把位置、现象、保留内容和屏幕宽度换成自己的问题。

换成自己的项目

代码块

- 1 我在【页面/设备宽度】发现【具体现象】，位置是【元素/区域】。
- 2 请先定位相关文件并解释可能原因；只修改【允许修改的范围】，不要改变【必须保持不变的功能】。
- 3 完成后说明在【验证宽度/操作路径】下的检查步骤。

实战步骤

1. 截取能看清问题的图片，隐藏私人信息。
2. 把图片和改写提示词一起发给 AI。
3. 先看 AI 提到的文件是否合理，再执行并复测。

完成后检查

问题在原设备宽度下消失；目标功能仍然可用；没有出现新的溢出或遮挡。

每日一问

你怎么描述那个问题，让 AI 一次就改对了（或改错了）？

Day 11 | 增加有反馈的互动效果

今天用时：45–60 分钟

[返回总目录](#) · [返回第 2 周](#)

学习目标

完成至少一个“用户操作后页面明确回应”的交互，例如收藏、筛选、表单提交、复制成功或展开详情。

注意：下面三个网站可以给交互和动效找灵感，都是选看、不是必需——motionsites、React Bits、3D Card Effect，按这三个名字搜索就能找到官网。挑一个看五分钟就够，不要在这里花掉整节课。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 11 | 第 2 周
- 2 今天要掌握：你加的反馈里，哪个动作让用户最明确地知道「生效了」？
- 3 今日板块顺序：① 交互反馈原理 ② 实现一个交互 ③ 连续操作测试
- 4 主任务：完成一个有反馈的交互
- 5 今日不做：依赖后端的交互（数据还没接）
- 6 完成标准：交互有可感知反馈（动效或提示）；连续操作不出错；改动已提交
- 7 今天交什么截图：点击前、点击后两张（或一段 3-5 秒录屏）——图里要有地址栏，以及点击引起的反馈
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 11 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：为按钮增加键盘操作和焦点状态
- 10 卡住降级：完成「复制成功」或「展开/收起」这类不依赖后端的交互

先讲清楚

交互反馈包括按钮状态、成功提示、失败提示、加载状态和空状态。反馈不是装饰，而是告诉用户操作有没有生效。

实战案例：今日热搜

代码块

- 1 请为“今日热搜”卡片增加收藏交互，但今天先使用前端临时状态，不接数据库。
- 2 用户点击收藏后，按钮文字和样式要变化，并显示“已收藏”；再次点击可以取消。
- 3 加载或处理期间按钮不可重复点击，发生错误时显示可理解的提示。
- 4 请保持现有卡片布局，先说明状态变化，再实现并给出成功、取消、失败三种测试步骤。

提示词怎么改

交互对象是卡片；状态变化是具体内容；“先用临时状态”是当天范围；禁止重复点击是边界；三种测试步骤是验证。把收藏换成你的动作和反馈文案。

换成自己的项目

代码块

- 1 请为【页面/组件】增加【交互动作】，今天先使用【临时状态/已有数据】。

- 2 用户操作后，界面要显示【成功反馈】；再次操作可以【撤销/修改】；处理期间要【禁用/显示加载】；失败时显示【失败提示】。
- 3 保持【不变的布局或功能】，先说明状态变化，再实现并给出【至少 3 种】测试步骤。

实战步骤

1. 先写出“操作前—处理中—成功—失败”的状态表。
2. 让 AI 小步实现，不要一次加入数据库。
3. 按正常、重复、失败三种路径测试。

完成后检查

至少一个交互有清晰反馈；按钮不会在处理中重复提交；失败时用户知道下一步怎么做。

每日一问

你加的反馈里，哪个动作让用户最明确地知道「生效了」？

Day 12 | 制作并使用自己的 Skill，完成筛选交互

今天用时：90–120 分钟

[返回总目录](#) · [返回第 2 周](#) · [查看 Skill 详解](#)

学习目标

创建一个能复用的前端设计 Skill，实际调用一次；再完成关键词、分类或等价条件筛选。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 12 | 第 2 周
- 2 今天要掌握：你写的 Skill 检查了什么？用一句话说。
- 3 今日板块顺序：① 理解 Skill 结构 ② 创建 Skill ③ 调用它完成筛选交互
- 4 主任务：创建并调用 Skill，完成一次筛选交互
- 5 今日不做：一天建多个 Skill；做与筛选无关的功能
- 6 完成标准：Skill 文件在项目内；Skill 至少真实调用一次并留有调用记录；筛选三种情况（有结果/无结果/清空恢复）测试通过
- 7 今天交什么截图：两张——筛选后的页面 + 项目里的 SKILL.md；图里要有地址栏、筛选条件与结果，以及 Skill 文件名
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 12 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：让 Skill 额外检查可访问性

10 卡住降级：工具不支持 Skill 就保存 frontend-guidelines.md，按同样方式手动调用一次，并提交等价证据

先讲清楚

Skill 是「可重复执行的工作说明」。WorkBuddy 学员按[附录 G](#)的方式创建项目内 Skill 文件。必做证据是：定义、调用记录、调用结果。

实战案例一：创建 Skill

代码块

- 1 请把当前项目已经确认的前端规则整理成可复用 Skill。
- 2 规则包含：页面层级、颜色和字体、卡片与按钮、移动端适配、修改前检查、修改后验证。
- 3 不要只写今日热搜专用内容；请告诉我文件位置、调用方法和 3 个成功标准。

提示词怎么改

背景是“当前项目已有规则”；具体内容是六类规则；“可复用”是迁移约束；文件位置、调用方法和成功标准是验证。改写时替换规则和下一个要调用 Skill 的任务。

代码块

- 1 请把【我的项目】中已经确认的【规则类别】整理为一个可复用 Skill。
- 2 它要帮助我完成【下一个任务】，遵守【项目约束】。
- 3 请告诉我 Skill 文件位置、调用方式和 3 个可观察的成功标准。

实战案例二：筛选交互

代码块

- 1 请在“今日热搜”列表增加关键词搜索和平台筛选。
- 2 用户输入关键词或选择平台后只显示匹配内容；没有匹配时显示“没有找到相关内容”；清空后恢复完整列表。
- 3 保持已经调用的设计 Skill，告诉我修改文件和正常、无结果、清空后三种测试步骤。

提示词怎么改

把“热搜列表、关键词、平台、空状态”换成自己的数据对象、筛选条件和提示文案；保持 Skill 是约束；三种测试是验证。

代码块

- 1 请在【我的页面】对【数据对象】增加【筛选条件】。
- 2 匹配时显示【结果规则】，无匹配时显示【空状态文案】，清空后【恢复规则】。
- 3 保持已调用的【Skill/规则】；告诉我修改了哪些文件和至少三种测试步骤。

实战步骤与检查

1. 保存 Skill，并在另一个组件或页面中调用一次。
2. 实现筛选，测试正常、无结果、清空三种情况。
3. 记录 Skill 调用前后提示词和结果差异。

每日一问

你写的 Skill 检查了什么？用一句话说。

Day 13 | 产出多级页面和四种可控状态

今天用时：45-60 分钟

[返回总目录](#) · [返回第 2 周](#)

学习目标

把项目扩展到至少 3 个页面或独立视图，并为列表/详情等页面准备加载、成功、空、错误四种状态。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 13 | 第 2 周
- 2 今天要掌握：页面之间怎么切换，你选了哪种方式？为什么？
- 3 今日板块顺序：① 规划视图结构 ② 实现多级切换 ③ 补齐四种状态
- 4 主任务：实现 3 个页面或独立视图，并为列表数据补齐四种状态
- 5 今日不做：路由库的进阶用法（够用就好）；新数据
- 6 完成标准：3 个视图互相切换不出错；空、加载、错误、正常四种状态都能看到；改动已提交
- 7 今天交什么截图：两张——视图切换 + 四种状态各一屏；图里要有地址栏，以及空、加载、错误、正常四种状态
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 13 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：加入返回上一页、面包屑或可访问的导航标签
- 10 卡住降级：路由配置困难就用三个独立区块做可点击切换，地址路由列入「下一步计划」，不阻塞后续任务

先讲清楚

路由是页面之间的地址或切换规则；状态是页面在不同情况下的表现。多页面不等于复制三张相似页面，而是每个页面承担不同任务。

实战案例：今日热搜

代码块

- 1 请根据 PRD 为“今日热搜”规划并实现 3 个可独立访问视图：首页、平台列表页、热搜详情页。
- 2 为列表数据补齐加载中、加载成功、没有结果、请求失败四种状态；每种状态都要有用户看得懂的反馈。
- 3 先列出页面路径和状态表，再逐页实现；不要新增登录和支付。

提示词怎么改

页面名称和路径是结构；四种状态是可控边界；“先列状态表”避免只做成功画面；不新增功能是范围控制。替换页面任务、路径、数据和不做事项。

换成自己的项目

代码块

- 1 请为【项目名称】规划并实现至少 3 个可独立访问视图：
- 2 【页面 1/路径】用于【任务】，【页面 2/路径】用于【任务】，【页面 3/路径】用于【任务】。
- 3 为【数据对象】准备加载、成功、空、错误四种状态，并写出每种状态的反馈文案。
- 4 先列页面路径和状态表，再逐页实现；不要新增【不做功能】。

实战步骤与检查

逐页打开路径；在正常、空数据和模拟失败时查看反馈；用手机宽度检查导航和长文字。

每日一问

页面之间怎么切换，你选了哪种方式？为什么？

[返回总目录](#)

Day 14 | 轻量用户测试和前端验收

今天用时：45-60 分钟

[返回总目录](#) · [返回第 2 周](#) · [进入第 3 周](#)

学习目标

把自己的项目仓库链接（或一段演示录屏）发给同伴，请同伴打开检查前端有没有问题。

（选做，适合有经验的同伴：同伴也可以在自己的 WorkBuddy 里打开你的项目让 AI 启动网站检查。主路径仍然是看仓库/录屏。）

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 14 | 第 2 周
- 2 今天要掌握：同伴卡在哪一步？这一步你自己为什么没发现？
- 3 今日板块顺序：① 准备测试清单 ② 真人轻量测试 ③ 最小修复
- 4 主任务：完成一次轻量用户测试和一次最小修复
- 5 今日不做：按反馈大改版；一口气全改
- 6 完成标准：一位真人按清单操作并给出反馈；至少完成一次修复；测试记录已存档
- 7 今天交什么截图：真人操作你页面的现场（对方设备屏幕）+ 你的测试记录——图里要有对方设备上打开的页面，以及你记下的那次修复
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 14 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：再测试一次手机宽度
- 10 卡住降级：自己间隔 10 分钟后按任务清单重新操作，记录第一次找不到的地方

先讲清楚

用户测试不是问“好不好看”，而是观察别人能不能完成任务。测试者不应被你提示下一步，否则你测到的是你的讲解能力。

实战步骤与检查

请一位同伴完成任务；记录他第一次点击、停顿和提问；只修复一个最大阻塞点，再让同伴复测。

每日一问

先对一下本课程的测试节奏：一共有三次对外测试，难度依次递增——第 2 周（今天）是同班同伴互测，第 3 周（Day 21）是同伴交叉验证，第 4 周（Day 27）请一位外部真人试用。今天这次的目标是发现明显卡点，不是把所有问题都找出来。

同伴卡在哪一步？这一步你自己为什么没发现？

今天同时是周验证日（第 2 周）：除了下面这一问和一张截图，还要按 [附录 A](#) 的第三张卡，多交一份完整材料。

第 3 周 | 后端、真实数据和云端数据库

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

注意：本周把「会动的页面」接上真实数据，并让成果可以发给别人打开。本周主线固定为：**先建数据模型，再写接口**。表结构先定下来，后面每个接口都在同一套字段上工作，不会每屏各造一套。

本周按这个顺序推进：

- **第 15 天**：先用一个不连数据库的健康接口 `/api/health`，打通「AI → CloudBase → 公网」的部署链路；同时把第 2 周的前端页面（mock 数据版）一并部署上线，并产出 `api-contract.md` 接口契约。
- **第 16 天**：建表和种子数据，也就是先确定数据长什么样。
- **第 17-19 天**：依次做读接口（GET）、写接口（POST），再把数据库代码从接口里拆出来做分层重构。
- **第 20 天**：把前端从 mock 数据切到真实接口并重新部署，完成公网检查台。
- **第 21 天**：第三周验收，产出云端数据服务 v1。

生词不用现在全懂，课上会逐个讲，[附录 C](#) 也能随时查。每个概念的详细做法见对应那一天的正文。

本周产出：一个公网可访问的「云端数据检查台」。再次说明：「今日热搜」只是老师贯穿使用的案例，你的项目主题自选；真实热搜数据接入是「今日热搜」案例的专属要求，自选项目的真实数据就是你自己产生的使用数据（打卡/待办记录），不强制接外部源；数据建模思路统一——按“每日计划 + 记录”两张表的模式设计（打卡应用示例：plan_days / checkins），表名按你的主题替换（如 todos / records）。

自选项目怎么建表

下面这张表按主题给出对应的两张表结构。找到最接近你主题的一行照做即可，不需要自己从零设计。

你的主题	表一（主对象）	表二（记录 / 关联）	核心字段示例
打卡 / 习惯	plan_days 每日计划	checkins 打卡记录	计划日、计划内容、完成状态、完成时间
待办清单	lists 清单	todos 待办项	标题、分类、截止日、是否完成
读书记录	books 书	reading_logs 阅读记录	书名、作者、页码、阅读日期、感想
课程管理	courses 课程	lessons 课时	课程名、教师、课时标题、进度、完成日
健身日志	exercises 动作	workout_logs 训练记录	动作名、组数、重量、日期

判断标准：如果你的主题找不到「一个主对象 + 一堆按时间累积的记录」这个结构，先按最接近的一行套用，跑通之后再调整——**不要在第 3 周重新设计。**

本周 Day 目录

[Day 15](#) · [Day 16](#) · [Day 17](#) · [Day 18](#) · [Day 19](#) · [Day 20](#) · [Day 21](#)

Day 15 | 打通云端链路：第一个云函数和接口契约

今天用时：90-120 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

提前说一件容易吃亏的事：CloudBase 的免费体验额度有到期时间，到期后你的公网网站会打不开，而且不会提前很久提醒你。所以今天开通时就顺手记下三样东西——环境 ID、剩余额度、到期日期——写进你的项目笔记。怎么查见[附录 M](#)和[附录 H](#)。

学习目标

按[附录 M](#)完成 CloudBase 注册并记录环境 ID、剩余额度、到期日期三项，在 AI 指引下开通云函数和静态托管；部署第一个不连数据库的云函数 `/api/health`，拿到公网地址；把第 2 周的前端页面（mock 版）部署到静态托管，拿到公网页面地址；同时产出 `api-contract.md` 接口契约，作为第 3 周建表和写接口的唯一依据。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 15 | 第 3 周
- 2 今天要掌握：公网地址第一次打开时，你第一个想确认的是哪一件事？
- 3 今日板块顺序：① 注册并开通 CloudBase（按附录 M，约 10-15 分钟） ② `/api/health` 云函数部署 ③ 前端 mock 版部署 ④ 接口契约
- 4 主任务：部署 `/api/health` 和前端 mock 版页面，并产出 `api-contract.md`
- 5 今日不做：真实业务接口、数据库建表、跨域配置（这些在 Day 16-20）
- 6 完成标准：`/api/health` 公网可访问返回 JSON；前端公网页面同伴手机能打开；`api-contract.md` 已放进仓库
- 7 今天交什么截图：三张——云函数公网地址的返回 + 前端公网页面 + 控制台环境信息；图里要有地址栏和返回的 JSON，以及环境 ID、剩余额度、到期日期
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 15 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 逐段解释云函数代码每一部分的作用
- 10 卡住降级：先保证 `/api/health` 公网可访问；前端部署受阻记下报错，Day 20 之前补齐即可

先讲清楚

第 15 天不写任何业务。先用一天证明一件事：AI 帮你写的东西，能真的部署到 CloudBase，并且被公网访问。这一步通了，后面 13 天的所有接口走的都是同一条路。同时，接口契约是前后端的「合同」：路径、字段、响应形状都写在这份文档里，后端照合同实现，前端照合同调用，谁也不去猜谁的代码。

先说清一个词：本周的可见作品叫「云端数据检查台」——就是在你第 2 周前端页面的基础上，让页面展示数据库真实数据、并保留一次写入测试入口。它不是新页面，就是你自己的项目上线后的样子。

实战案例：今日热搜

演示：新建一个最小云函数，只有一个 GET `/api/health`，返回 `{ok:true}`；部署后用浏览器直接打开公网地址，看到返回结果。再打开 `api-contract.md`，指出本项目后面所有接口（列表、收藏、修改、删除）都先在这里登记占位，今天一个都不实现。

提示词怎么改

让 AI 生成的云函数必须「最小」：不连数据库、不写业务，只返回健康状态；必须同时给出部署步骤和浏览器验证方法。契约文档要写明每个接口的路径、方法、请求参数、响应 JSON 形状和错误形状，一处都不要省。今天只登记占位，不要求实现。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例创建第一个 CloudBase 云函数并给出部署步骤。
- 2
- 3 要求：
- 4 - 只实现 GET `/api/health`，不连数据库、不写任何业务逻辑；
- 5 - 返回 JSON：{ "ok": true, "service": "hot-search-demo", "time": "<服务器时间>" }；
- 6 - 给出从创建到部署到公网访问的完整步骤；
- 7 - 给出部署后的验证方法（浏览器打开公网地址应看到什么）。
- 8
- 9 另外给出前端部署步骤：
- 10 - 构建当前 React 项目（mock 数据版）并上传到 CloudBase 静态托管；
- 11 - 给出部署后的验证方法（浏览器打开公网地址应看到页面）；
- 12 - 暂时不处理跨域，接口还没有接。
- 13
- 14 同时生成 `api-contract.md` 初稿，登记本项目后续所有接口：
- 15 - GET `/api/hot`：返回热搜列表；
- 16 - GET `/api/favorites`：返回收藏列表；
- 17 - POST `/api/sync`：从公开热搜来源拉取当日真实热搜写入 `trends` 表（第 17 天实现，手动触发；本课程不做定时自动同步）；

- 18 - POST /api/favorites: 新增收藏;
- 19 - PATCH /api/favorites/:id: 修改收藏备注 (第 4 周实现);
- 20 - DELETE /api/favorites/:id: 取消收藏 (第 4 周实现)。
- 21 每个接口写明: 路径、方法、请求参数、响应 JSON 形状、错误时返回什么。今天只登记占位, 不实现。

换成自己的项目

代码块

- 1 请为我的项目【项目名称】创建第一个 CloudBase 云函数并给出部署步骤。
- 2
- 3 要求:
- 4 - 只实现 GET /api/health, 不连数据库、不写业务逻辑;
- 5 - 返回 { "ok": true, "service": "【项目英文名】" };
- 6 - 给出完整部署步骤和浏览器验证方法。
- 7
- 8 另外给出前端部署步骤:
- 9 - 构建第 2 周的 React 项目 (mock 数据版) 并上传到 CloudBase 静态托管;
- 10 - 给出部署后的验证方法 (浏览器打开公网地址应看到页面);
- 11 - 暂时不处理跨域, 接口还没有接。
- 12
- 13 同时基于我第 2 周的前端页面生成 api-contract.md:
- 14 - 列出前端需要后端提供的全部接口 (打卡应用示例: plan_days / checkins 两张表对应的读写接口; 其他项目从你的页面需求推导自己的表和接口, 别忘了列表读取接口—案例是 GET /api/favorites, 自选项目是记录表读取);
- 15 - 每个接口写明: 路径、方法、请求参数、响应 JSON 形状、错误返回;
- 16 - 这份契约是第 3 周建表和写接口的唯一依据, 今天只登记占位, 不实现。

实战步骤

1. 今天才需要 CloudBase: 按[附录 M](#)《CloudBase 注册与使用指导手册》完成注册, 顺手记下环境 ID、剩余额度、到期日期三项。云函数和静态托管的开通, 由 AI 给出逐步指引, 你在控制台逐步确认完成——每一步都需要你亲自点确认, AI 不能代点。
2. 让 AI 生成 `/api/health` 云函数并部署, 用浏览器访问公网地址, 确认返回 JSON。
3. 让 AI 给出前端构建与部署步骤: 构建第 2 周的 React 项目并上传到静态托管, 用浏览器打开公网页面地址, 确认页面能打开 (此时展示的是 mock 数据, 属正常)。
4. 把前端公网链接发给一名同伴, 请对方打开确认。
5. 先和 AI 一起填一张两列表格再生成契约: 左列列出第 2 周页面上的每个动作 (浏览列表、筛选、收藏、写备注……), 右列填这个动作需要后端提供什么接口。填完这张表, 把它交给 AI, 让它据此生成 `api-contract.md` 初稿, 你逐行核对后确认, 登记第 3 周要实现的全部接口。

6. 把契约文档放进项目仓库，作为本周唯一的仲裁物。
7. 截图保存：控制台云函数列表 + 静态托管页面 + 公网访问结果。

完成后检查

- `/api/health` 公网地址浏览器可直接访问，返回 `{ok:true}`。
- 前端 mock 版公网页面可访问（同伴手机能打开）。
- `api-contract.md` 已生成，包含路径、方法、响应形状和错误形状。
- 明确知道自己项目用哪几张表（打卡应用示例：plan_days / checkins）。
- 部署截图和公网链接已保存。

今天怎么检测

1. **部署链路检测**：浏览器直接打开 `/api/health` 的公网地址，应看到 `{"ok":true,...}` 的 JSON。看到 = 链路通；打不开 = 部署没成功。（截图存证）
2. **契约完整性检测**：把 `api-contract.md` 发给 AI，输入：「逐项核对这份契约，每个接口是否写全了路径、方法、请求参数、响应形状、错误形状？缺哪项直接指出。」AI 能挑不出缺失项 = 合格。
3. **召回检测**：不看文档，口述回答三个问题——我的项目用哪几张表？第一个要实现的接口是什么？接口出错时返回什么形状？答不上来的部分就是今天没吃透的部分。
4. **前端公网检测**：把前端公网地址发给同伴或用手机流量打开，能看到第 2 周的页面 = 部署链路前后端都通了。（截图存证）

自检留档（群内只交一张截图）：健康接口公网地址 + 截图、前端公网页面地址 + 截图、契约文档链接、召回三问的答案。

每日一问

公网地址第一次打开时，你第一个想确认的是哪一件事？

Day 16 | 数据模型先行：建表和种子数据

今天用时：45-60 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

在写任何业务接口之前，先定下数据模型：建表 + 可重复执行的种子脚本。示例使用 plan_days / checkins 两张表，实际表名从你的 api-contract.md 推导。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 16 | 第 3 周
- 2 今天要掌握：你的两张表分别存什么？它们靠哪个字段关联？
- 3 今日板块顺序：① 数据模型设计 ② 建表 ③ 种子脚本 ④ select 验证
- 4 主任务：建表 + 种子脚本 + select 验证
- 5 今日不做：写任何接口（Day 17 才开始）；改前端
- 6 完成标准：每张核心表 select 至少 5 行；seed.sql 重复执行不报错；db/schema.sql 和 seed.sql 已入库
- 7 今天交什么截图：CloudBase 控制台数据库的表数据页——图里要有表名，以及每张核心表至少 5 行数据
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 16 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：给字段加注释，让 AI 解释每个字段类型为什么这么选
- 10 卡住降级：先保证一张表 + 种子能跑通；另一张在 Day 18 之前补上即可，Day 17 的读接口只依赖你的核心表

先讲清楚

数据模型是应用的「宪法」。表结构先定下来，后面 AI 写的每一个接口都在同一套字段上工作，不会每屏各造一套。今天不写接口，只做两件事：建表、灌入示例数据（种子），并且脚本可以重复执行（先删后建再插入），随时能推倒重来。先定地基，再盖房子。seed.sql 只用来验证「可重复执行」这项能力；第 17 天接入真实热搜数据后，页面主数据不再依赖 seed。注意：seed.sql 只在开发阶段使用；第 20 天部署上线后禁止再执行，否则会清空真实数据——上线后的数据变更一律用增量 SQL；另外说明一点，本课程不登录、不建用户表，表里也没有「身份 / 用户」字段——唯一约束只加在业务字段上（案例是「来源平台 + 标题 + 日期」）。

实战案例：今日热搜

演示：为案例创建 trends（热搜条目）和 favorites（收藏）两张表，写一份「先删后建再插入」的种子脚本，在控制台执行两次都成功；用 select 看到真实的数据行。注意：这两张表只是案例演示，不要照抄到你的项目里。

提示词怎么改

建表提示词必须让 AI「从契约推导表结构」，而不是自由发挥：字段从 api-contract.md 来，不要自己另造一套。种子脚本必须可重复执行，并要求 AI 给出执行步骤和 select 验证方法。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例设计数据模型并生成可重复执行的 SQL 脚本。

```
2
3 表结构（案例专用）：
4 - trends: id、标题、热度、来源平台、排名、日期、抓取时间；
5 - favorites: id、trends_id（关联 trends）、备注、创建时间。
6
7 要求：
8 - 生成 schema.sql：建表语句，含主键、外键，以及（来源平台，标题，日期）的唯一索引；
9 - 生成 seed.sql：先 DROP 再 CREATE 再 INSERT 至少 5 条示例数据，保证重复执行不报错；
10 - 给出在 CloudBase PostgreSQL 控制台执行的步骤和验证方法（用 select 查看插入的行）；
11 - 简单说明每个字段为什么选这个类型。
12 注意：这两张表只是案例演示，不要直接用于你的项目。
```

换成自己的项目

代码块

```
1 请根据我的 api-contract.md 为项目【项目名称】设计数据模型并生成 SQL 脚本。
2
3 要求：
4 - 表结构从你的 api-contract.md 推导，不要自己另造一套（打卡应用示例：plan_days（每日计划）、checkins（打卡记录））；
5 - 生成 schema.sql（建表，含主键、外键、必要约束）和 seed.sql（先删后建再插入至少 5 条可复现的示例数据）；
6 - 脚本必须可重复执行；
7 - 给出在控制台执行和用 select 验证的步骤；
8 - 表结构确定后回写到 api-contract.md，保持契约与数据库一致。
```

实战步骤

1. 对照 `api-contract.md`，让 AI 列出需要的表和字段。
2. 让 AI 生成建表 SQL 和可重复执行的种子脚本。
3. 在 CloudBase PostgreSQL 控制台执行，用 select 验证数据行真实存在。
4. 把 SQL 文件保存进项目仓库（如 `db/schema.sql`、`db/seed.sql`）。

完成后检查

- 表已创建，字段和 `api-contract.md` 一致。
- 种子脚本重复执行不报错。
- 能用一条 select 看到真实数据行。
- SQL 文件已提交进仓库。

今天怎么检测

1. **数据可见检测**：在控制台执行 `select * from 你的核心表名`；（表名建表时定过，打开 `db/schema.sql` 就能看到；打卡应用示例是两张表），每张各返回至少 5 行种子数据。有行 = 表真实存在且数据灌进去了。（截图存证）
2. **可重复执行检测**：把 `seed.sql` 从头再执行第二遍。不报错 = 脚本可重复执行合格；报错 = 脚本里缺了「先删后建」。
3. **契约一致性检测**：把表结构和 `api-contract.md` 发给 AI，输入：「逐字段核对我的表结构和契约文档里的数据形状，指出不一致的地方。」挑出不一致 = 合格。
4. **仓库留痕检测**：在仓库里能找到 `db/schema.sql` 和 `db/seed.sql` 两个文件，打开能看到完整 SQL。

自检留档（群内只交一张截图）：两条 select 结果截图、第二遍执行成功的截图、SQL 文件链接。

每日一问

你的两张表分别存什么？它们靠哪个字段关联？

Day 17 | 第一条读取接口：GET

今天用时：60-75 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

写第一批连接数据库的云函数：GET 读接口（案例两个：GET /api/hot、GET /api/favorites），并接入真实热搜数据，按契约统一返回 {ok, data, error} 形状。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 17 | 第 3 周
- 2 今天要掌握：接口返回的数据里，哪一项和你建的表对不上？怎么发现的？
- 3 今日板块顺序：① GET 读接口实现 (/api/hot 和 /api/favorites) ② 接真实热搜数据（同步任务） ③ 部署 ④ 真库验证
- 4 主任务：GET 读接口部署并完成验证；真实热搜数据入库且接口能读出
- 5 今日不做：业务写入接口（收藏写入留到 Day 18）；改表结构
- 6 完成标准：公网打开 GET /api/hot 返回当日真实热搜 {ok:true,data:[...]}; GET /api/favorites 返回收藏列表；控制台改一行数据，刷新后返回跟着变
- 7 今天交什么截图：两张——GET 接口公网地址的返回 + 页面上显示的真实数据；图里要有地址栏，以及真实热搜条目（不是假数据）

- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 17 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：加一个简单查询参数（如返回条数限制）
- 10 卡住降级：真实数据源不可用时按附录 F 切备用源，仍不通就保留 seed 并在页面标注「示例数据」，记入待办；接口本身先在控制台验证 SQL 正确，再排查部署环节

先讲清楚

从今天起，前端要的数据不再来自写死的 mock，而来自真实数据库。第一个接口只做一件事：读。响应形状统一为 `{ok, data, error}`：成功时 `ok` 为 true、数据放在 `data` 里；出错时 `ok` 为 false、`error` 里放人能看懂的中文说明。前端以后只认这一个形状，所有接口都不例外。

实战案例：今日热搜

演示：写 GET `/api/hot` 云函数，查询 `trends` 表，按热度倒序返回前 20 条；浏览器打开公网地址，看到 JSON 里是数据库里的真实数据；改一条数据库记录，再请求一次，数据跟着变——证明读的是真库，不是写死的。

提示词怎么改

提示词里必须给出响应形状 `{ok, data, error}`，并要求 AI 同时给出「怎么在浏览器里验证」的步骤；SQL 必须参数化，禁止字符串拼接。再加一个验证要求：给一个「修改数据库数据后确认接口返回值变化」的方法。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例实现第一个读取接口并给出部署步骤。
- 2
- 3 要求：
- 4 - 云函数实现 GET `/api/hot`：查询 `trends` 表，按热度倒序返回前 20 条；
- 5 - 响应统一形状：成功 `{ "ok": true, "data": [...] }`；失败 `{ "ok": false, "error": "人能看懂的中文说明" }`；
- 6 - SQL 必须参数化，禁止字符串拼接；
- 7 - 给出部署步骤和浏览器验证方法；
- 8 - 给出一个验证“读的是真数据库”的方法：修改一条数据库记录后重新请求，确认返回值变化；
- 9 - 另外生成数据同步云函数 POST `/api/sync`：三个平台的具体来源、必需请求头、字段映射与判重规则见 附录 F 的「同步提示词模板」，按它的要求实现；不绕过任何反爬、登录或频率限制；给出手动触发方法和验证步骤。

换成自己的项目

```
1 请根据 api-contract.md 为项目【项目名称】实现第一个读取接口。
2
3 要求：
4 - 实现契约中登记的 GET 接口：从你的核心表和记录表读取（打卡应用示例：plan_days 和
  checkins 各一个 GET 接口）；
5 - 响应统一 { ok, data, error } 形状，与契约完全一致；
6 - SQL 参数化；
7 - 给出部署步骤、浏览器验证方法，以及"改一条数据库数据、确认接口跟着变"的验证步骤；
8 - 实现完成后，把该接口在 api-contract.md 中标记为“已实现”。
9
10 【动手前先回答三个问题：我的数据从哪来】
11 1. 这份数据是「别人产出的」还是「我的用户产出的」？（热搜、天气 = 别人的；打卡记录、待办、
  读书笔记 = 用户的）
12 2. 我要「只看当前」，还是「要能回看过去」？
13 3. 这份数据明天还在原地吗？（公开接口一直都在；用户昨天的输入不存就没了）
14
15 按结论选路：
16 - 别人产出 + 不用回看历史 → 实时拉取，不建表（例如天气、汇率）
17 - 别人产出 + 要回看历史 → 定期拉取入库（「今日热搜」案例就是这一类）
18 - 用户产出 → 必须存进数据库，这是你项目的核心资本
19
20 对大多数同学：你的「真实数据」不是外部接口，而是你自己数据库里的用户数据。今天要做的是让接
  口从你自己建的表里读出真实数据，
21 不需要去找外部 API。只有当项目确实要展示外部信息（天气、课程表、汇率）时，才按附录 F 的来
  源要求去找合法公开接口。
```

实战步骤

1. 让 AI 按契约生成 GET 云函数。
2. 部署后浏览器访问公网地址，确认返回 `{ok:true,data:[...]}`。
3. 改一条数据库数据，再请求一次，确认数据变化。
4. 故意制造一个错误（如表名写错），确认返回 `{ok:false,error}`，不是白屏或 500 裸报错。

完成后检查

- GET 接口公网可访问，返回形状与契约一致。
- 数据来自真实数据库（改库后接口数据跟着变）。
- 错误时返回 `{ok:false,error}`，错误信息人能看懂。
- 契约文档中该接口已标记"已实现"。

今天怎么检测

1. **读取检测**：浏览器打开 GET 接口公网地址，应看到 `{ok:true,data:[...]}`，且 `data` 是当日真实热搜数据（和附录 F 选定来源的当前榜单对得上），不是种子数据。（截图存证）
2. **真库检测（最关键）**：在控制台把某一行数据改一个字（如把某条打卡状态改掉），刷新接口页面——返回值跟着变 = 读的是真库；不变 = 读的是写死的假数据。
3. **错误形状检测**：故意把代码里表名改错、重新部署、再请求一次，应返回 `{ok:false,error:"..."}` 且 `error` 是中文说明，不是白屏或英文堆栈。测完改回来。
4. **契约对齐检测**：把接口实际返回的 JSON 和 `api-contract.md` 里登记的形状发给 AI，输入：「对比这两者，字段名、层级、类型是否完全一致？」不一致就是违约，必须修。

自检留档（群内只交一张截图）：正常返回截图、改库前后接口对比截图、错误响应截图。

每日一问

接口返回的数据里，哪一项和你建的表对不上？怎么发现的？

Day 18 | 第一条业务写入接口：POST

今天用时：45–60 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

写第一个业务写入接口：POST，把提交的数据真实写进数据库，并处理校验、防重复和错误三种情况。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 18 | 第 3 周
- 2 今天要掌握：你防了哪一种重复提交或错误输入？怎么测的？
- 3 今日板块顺序：① POST 接口实现 ② 部署 ③ 写入与读回验证
- 4 主任务：POST 接口部署并完成一次真实写入 + 读回验证
- 5 今日不做：PATCH 和 DELETE（第 4 周）；批量写入
- 6 完成标准：正常 POST 返回 `{ok:true,...}`，形状与 `api-contract.md` 一致，且数据库多一行；重复提交被拒；缺必填字段被拒且提示是中文
- 7 今天交什么截图：两张——POST 成功返回 + 数据库里新增的那一行；图里要有返回的 JSON 形状，以及表里新增的记录
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 18 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：加一条服务端日志，方便以后排查问题

先讲清楚

能读只是「看」，能写才是「用」。今天要打通「提交 → 云函数校验 → 写入数据库 → 再读出来确认」的完整闭环。写入比读取多三件事：校验（缺字段、超长要拒绝）、防重复（同一数据提交两次怎么办）、错误处理（失败时告诉用户为什么）。这三件事今天一次做齐，后面所有写接口都照这个标准。

实战案例：今日热搜

演示：写 POST `/api/favorites` 云函数，把一条热搜加入收藏写入 `favorites` 表；用调试工具发送请求，再到数据库里 `select` 看到新行；重复提交同一条，观察防重复逻辑；缺字段提交，看到明确的错误信息。

提示词怎么改

写入提示词必须写清楚：必填字段、字段类型和长度限制、重复的判定标准、出错时返回什么。可以让 AI 先给一份校验清单再写代码。最后要求给出三条测试命令：正常写入、重复提交、缺字段提交。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例实现第一个业务写入接口（写收藏）并给出部署步骤。
- 2
- 3 要求：
- 4 - 云函数实现 POST `/api/favorites`：接收 `trends_id` 和备注，写入 `favorites` 表；
- 5 - 写入前校验：必填字段缺失、类型错误、超长文本要拒绝，并返回中文错误说明；
- 6 - 防重复：同一 `trends_id` 已在收藏中时返回明确错误，不重复写入；
- 7 - 响应统一 `{ ok, data, error }`；
- 8 - 给出用调试工具测试的三条命令：正常写入、重复提交、缺字段提交；
- 9 - 给出写入后在数据库里 `select` 验证新行的方法。

换成自己的项目

代码块

- 1 请根据 `api-contract.md` 为项目【项目名称】实现写入接口。
- 2
- 3 要求：
- 4 - 实现契约中登记的 POST 接口：向你的核心表写入一条记录（打卡应用示例：`checkins`）【必填字段按契约来，如所属计划日、完成状态、日期】；
- 5 - 写入前校验必填字段和格式，错误信息用中文写清楚缺了什么；

- 6 - 防重复：同一天同一计划项重复打卡时按合同约定处理【拒绝或返回已存在】；
- 7 - 响应统一 { ok, data, error }；
- 8 - 给出三条测试命令（正常 / 重复 / 缺字段）和数据库 select 验证方法；
- 9 - 最后再调一次读取接口（GET /api/favorites），确认新写入的数据能被读出来。

实战步骤

1. 让 AI 按契约生成 POST 云函数（含参数校验）。
2. 部署后用调试工具发送一条正常请求，数据库 select 确认新行。
3. 测试重复提交和缺字段提交，确认都被拒绝且错误信息能看懂。
4. 再调一次 GET 接口，确认新写入的数据能被读出来——读写闭环成立。

完成后检查

- 正常写入成功，数据库有真实新行。
- 重复和缺字段请求被拒绝，错误信息人能看懂。
- 写入后 GET 能读到新数据（读写闭环成立）。
- 数据库记录截图已保存。

今天怎么检测

1. **写入检测**：用调试工具发送正常 POST 请求 → 返回 {ok:true}；立刻在控制台执行 `select * from 你的写入表 order by id desc limit 1;`（表名建表时定过，打开 `db/schema.sql` 就能看到）→ 最顶上就是你刚写的那一行。（写入前后两张截图对比）
2. **防重复检测**：把同一条请求原样再发一次 → 应被拒绝并返回明确说明，且数据库行数没有增加。
3. **校验检测**：删掉一个必填字段再发 → 应被拒绝，错误信息能说出「缺了什么」，而不是笼统的报错。
4. **闭环检测**：调一次 GET /api/favorites → 刚才写入的数据出现在返回里。写进去 + 读出来，两端都看到 = 闭环成立。
5. **行数核对检测**：记录写入前后的行数（写入前 N 行，正常写入 +1，重复提交仍 N 行）——三个数字对得上 = 校验和防重复都真的在工作。

自检留档（群内只交一张截图）：写入前后 select 对比截图、重复/缺字段被拒截图、GET 读回截图。

每日一问

你防了哪一种重复提交或错误输入？怎么测的？

Day 19 | 数据访问层：把数据库代码从接口里拆出来

今天用时：45-60 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

把散在接口里的数据库查询代码，拆成独立的数据访问层（Repository）：接口只管「接请求、返响应」，查询只出现在 repository 文件里。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 19 | 第 3 周
- 2 今天要掌握：拆完之后，「查数据库」这段代码从哪移到了哪？
- 3 今日板块顺序：① 理解分层思路 ② 拆出数据访问层 ③ 全接口回归
- 4 主任务：完成一次完整重构 + 全接口回归验证
- 5 今日不做：改接口路径和字段名（契约不许动）；加新功能
- 6 完成标准：数据库操作已拆成独立文件；所有已上线接口回归通过；改动已提交
- 7 今天交什么截图：两张——项目的文件结构 + 重构后接口仍正常返回；图里要有新拆出来的数据库操作文件，以及地址栏和接口返回
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 19 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：让 AI 画一张分层示意图放进项目文档
- 10 卡住降级：只拆一个接口的一个查询，跑通后再拆第二个

先讲清楚

前几天为了先跑通，数据库查询是直接写在接口函数里的。能跑，但每加一个接口就抄一遍，以后改表字段要改 N 处。今天不写新功能，专门做一次重构：把「怎么查数据库」收进 repository 文件，接口里只留一行调用。这就是分层——入口层管 HTTP、业务层管逻辑、数据访问层管 SQL。先跑通再重构，比一开始就追求完美分层更适合零基础：你先见过「乱但能跑」的样子，才知道拆什么、为什么拆。

实战案例：今日热搜

演示：把 GET `/api/hot` 里查 `trends` 的 SQL 抽到 `trendsRepository.js`，接口里变成一行调用；功能不变，每个接口重新调用一遍，结果照旧一致；再看第二个接口怎么复用同一个文件。

提示词怎么改

重构提示词必须带一条硬要求：「重构后所有接口行为不变」，并附回归验证方式（每个接口重新调一遍，对比结果）。明确禁止 AI 在重构时顺手加新功能、改响应形状。

提示词参考：今日热搜

代码块

- 1 请对「今日热搜」案例的云函数代码做一次重构，把数据库查询拆成数据访问层。
- 2
- 3 要求：
- 4 - 新建 trendsRepository.js 和 favoritesRepository.js，把所有 SQL 查询移进去，只对外暴露函数；
- 5 - 接口文件里不再出现任何 SQL，只调用 repository 提供的函数；
- 6 - 所有接口的行为必须完全不变；
- 7 - 给出重构后的目录结构说明（入口 / 业务 / 数据访问各放什么）；
- 8 - 给出回归验证清单：每个接口重新调用一遍，对比重构前的返回结果，逐项确认一致。
- 9 不要在重构时新增任何功能。

换成自己的项目

代码块

- 1 请对项目【项目名称】的后端代码做同样的分层重构。
- 2
- 3 要求：
- 4 - 新建核心表的 repository 文件（打卡应用示例：checkinsRepository.js 和 planDaysRepository.js，按你的实际表命名）；
- 5 - 所有数据库查询集中到 repository 文件，接口只保留"接请求、调函数、返响应"；
- 6 - 重构后所有接口行为不变，响应形状不变；
- 7 - 给出回归验证清单：GET、POST 等已有接口逐个重新调用，确认结果与重构前一致；
- 8 - 给出重构后的目录结构说明。
- 9 不要新增任何功能。

实战步骤

1. 让 AI 把数据库查询代码抽成 repository 文件。
2. 逐个接口重新部署、重新调用，确认行为不变。
3. 对比重构前后代码，确认接口文件变薄、SQL 集中。
4. 截图保存目录结构（入口 / 业务 / 数据访问）。

完成后检查

- 所有接口重构后行为不变（逐个验证过）。
- 数据库查询只存在于 repository 文件里。
- 接口文件里看不到任何 SQL。

· 目录结构截图已保存。

今天怎么检测

1. **分层到位检测**：把这句话发给 AI 代查并截图：「请在接口文件里搜索 select、insert、update 等 SQL 关键字，再在 repository 文件里搜索同样的关键字，分别报告命中次数和位置」→ 接口文件命中 0 次、repository 文件全部命中 = 分层到位。（两张搜索截图）
2. **行为不变检测（回归）**：重构前先截图保存每个接口的返回；重构后逐个重新请求，一张张对比——内容、字段、条数完全一致 = 重构没有破坏任何东西。这是今天最重要的检测。
3. **新功能检测**：对比重构前后的功能——如果多了任何昨天没有的能力，说明 AI 违规加了功能，必须删掉。重构只许「搬家」，不许「添家具」。
4. **结构可讲检测**：让 AI 生成目录结构说明后，自己合上文档回答：「入口层放什么？数据访问层放什么？为什么接口里不许写 SQL？」答不出 = 分层只是搬了代码，没进脑子。

自检留档（群内只交一张截图）：关键字搜索截图、回归对比截图、目录结构截图。

每日一问

拆完之后，「查数据库」这段代码从哪移到了哪？

Day 20 | 部署到公网：云端数据检查台

今天用时：90-120 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

把前端从 mock 数据切换到真实接口（接 API、处理跨域），重新构建并上传到静态托管（流程与第 15 天完全相同），得到展示真实数据的公网检查台。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 20 | 第 3 周
- 2 今天要掌握：跨域那一下，你是怎么认出问题出在哪的？
- 3 今日板块顺序：① 确认云函数可用并配置跨域 ② 本地接线 ③ 重新构建上传 ④ 逐项验证
- 4 主任务：拿到一个可分享的公网 URL，页面展示真实数据
- 5 今日不做：从零重新部署（流程 Day 15 走过）；改后端代码
- 6 完成标准：公网首页展示数据库真实数据；控制台改数据刷新跟着变；F12 里请求地址是公网地址
- 7 今天交什么截图：公网首页——图里要有完整的公网 URL（地址栏），以及页面上显示的数据库真实数据

- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 20 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：给检查台加一个「最后更新时间」显示
- 10 卡住降级：优先保证健康接口 + 首页可打开；数据展示问题记录下来明天处理

先讲清楚

前端在第 15 天已经把 mock 版搬上公网，今天做的是换心脏：把页面请求从 mock 切换到真实接口，再重新构建上传一次——部署流程和第 15 天完全一样，不是新知识。检查台页面做的事很简单：展示真实数据（案例是热搜榜单，自选项目是你的核心记录）、展示接口健康状态，让任何人打开链接就能确认“数据是真的、链路是通的”。这是本周最重要的产出，也是第 4 周一切工作的底座——后面 8 天都在这个地址上改，不再从零部署。

提前给你一份跨域（CORS）预置答案，这是今天最高频的卡点：①报错长什么样——浏览器控制台出现红色报错，关键词是 `Access-Control-Allow-Origin`，页面能打开但数据是空的；②去哪配——CloudBase 控制台 → 云函数 → HTTP 访问服务 → 跨域配置，把你自己的静态托管域名加进去（不要填 * 通配符）；③怎么确认配好——刷新页面能看到真实数据，F12 Network 里那条请求没有红色 CORS 报错。第一次用开发者工具的，先看[附录 K](#)。

实战案例：今日热搜

演示：把前端从 mock 切换到真实接口并重新构建上传（流程与第 15 天相同）；打开公网首页，页面上是数据库里的真实热搜；把链接发给同伴，对方用自己的手机也能打开。

提示词怎么改

部署提示词分两步说清：先部署后端（云函数 + HTTP 访问），再部署前端（构建 + 静态托管），并要求 AI 给出「部署后逐项验证清单」。跨域（CORS）是今天最常见卡点，提示词里直接让 AI 预先处理，并提前列出今天最可能遇到的卡点。

提示词参考：今日热搜

代码块

- 1 请把「今日热搜」前端从 mock 数据切换为调用公网接口，并重新部署。
- 2
- 3 第一步（本地接线）：
 - 4 - 把页面请求从 mock 数据切换为按 `api-contract.md` 调用公网接口地址；
 - 5 - 处理跨域（CORS）：在 CloudBase 控制台云函数 HTTP 访问服务的跨域配置里，只允许我的静态托管域名，禁止使用 * 通配符；
 - 6 - 先在本地把页面调通，确认显示数据库真实数据。
- 7
- 8 第二步（重新部署）：
 - 9 - 本地调通后，重新构建并上传到 CloudBase 静态托管（流程与第 15 天完全相同）；
 - 10 - 给出部署后逐项验证清单（首页可打开、数据来自数据库、接口可调用）；

- 11 - 检查台页面展示：健康状态、真实热搜列表、一次收藏写入测试入口；
- 12 - 提前列出今天最可能的 3 个卡点（跨域报错、环境变量未生效等）及其处理方法。
- 13
- 14 第三步（收尾，必须做）：
- 15 - 移除页面上所有「Mock」「示例数据」字样（包括页脚说明和卡片上的更新提示），更新时间改为显示接口返回的真实抓取时间；
- 16 - 标题下的副标题写明本页聚合的三个平台是微博、抖音、B站。
- 17
- 18 最终页面应该长这样（用这段文字描述代替"看图对照"）：
- 19 页面顶部是标题「今日热搜」和一句副标题（写明聚合微博、抖音、B站三大平台），右上角显示数据更新时间；
- 20 主体是三个平台并列的榜单区域，每个平台一个卡片栏，栏头有平台名和榜单名，栏内是该平台当天热搜的若干条，
- 21 每条显示序号、标题和热度（序号在前、热度在右）；榜单里的数据全部来自数据库中的真实数据，
- 22 页面上不出现任何"示例数据""Mock"字样。

换成自己的项目

代码块

- 1 请把我的项目【项目名称】前端从 mock 数据切换为调用公网接口，并重新部署。
- 2
- 3 第一步（本地接线）：
- 4 - 把页面请求从 mock 数据切换为按 api-contract.md 调用公网接口地址；
- 5 - 处理跨域（CORS）：只允许我自己的静态托管域名，禁止使用 * 通配符；
- 6 - 先在本地把页面调通，确认显示数据库真实数据（打卡应用示例：checkins 表）。
- 7
- 8 第二步（重新部署）：
- 9 - 本地调通后，重新构建并上传到静态托管（流程与第 15 天完全相同）；
- 10 - 给出逐项验证清单，以及把链接发给同伴请对方打开的验证说明；
- 11 - 检查台展示：健康状态、你的核心表真实数据、一次写入测试入口；
- 12 - 所有密钥走环境变量，确认部署配置里没有硬编码；
- 13 - 提前列出最可能的卡点（跨域、环境变量、构建报错）和处理方法。

实战步骤

1. 让 AI 给出接线和重新部署的完整步骤，按“本地接线”“重新部署”两段依次执行。
2. 确认所有云函数和 HTTP 访问路径公网可用（第 15–19 天已部署过，这里是核对），并在控制台完成跨域（CORS）配置：只允许自己的静态托管域名。
3. 重新构建并上传前端到静态托管（构建、上传命令与第 15 天相同），打开公网首页。让 AI 指出接口地址定义在哪个文件或环境变量，确认构建前已指向公网地址。
4. 确认页面展示的是数据库真实数据，不是 mock；从今天起不要再执行 seed.sql（见第 16 天的提醒）。

5. 把公网链接发给一名同伴，请对方用自己的设备打开确认。

完成后检查

- 公网首页可访问，展示真实数据库数据。
- 健康接口和读写接口在公网都可用。
- 至少一名同伴从自己的设备打开成功。
- 跨域等部署问题已解决并有记录。

今天怎么检测

1. **公网可达检测**：把公网链接发给同伴（或用自己手机流量、关掉电脑操作），对方能打开且看到数据 = 真公网。只有自己电脑能开 = 可能部署没成功。（同伴确认记录存证）
2. **真数据检测**：在控制台改一条数据，刷新公网页面——页面上的内容跟着变、且热搜榜与[附录 F](#)来源的当前榜单对得上 = 页面连的是真库、用的是真实数据；不变 = 页面还在用 mock，部署只成功了一半。
3. **请求去向检测**：在页面上按 F12 打开 Network 面板（不熟悉开发者工具的，先看[附录 K](#)），刷新一次——接口请求的地址必须是公网地址（`https://...`），出现 `localhost` 或 `127.0.0.1` 就是没切换干净。
4. **三件套检测**：公网首页可打开、健康接口返回 `ok:true`、页面上完成一次写入并刷新仍在——三件全过 = 第 3 周的底座立住了。

自检留档（群内只交一张截图）：公网 URL、真实数据页面截图、F12 请求地址截图、同伴打开确认记录。

每日一问

跨域那一下，你是怎么认出问题出在哪的？

Day 21 | 第三周验收：云端数据服务 v1

今天用时：45–60 分钟

[返回总目录](#) · [返回第 3 周](#) · [本周 Day 目录](#)

学习目标

用 PASS / FAIL / 未执行 清单验收第 3 周，完成一次 3–5 分钟演示提纲，并完成每周至少一次的[同伴交叉验证](#)。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 21 | 第 3 周
- 2 今天要掌握：这周最花时间的是哪一步？你觉得值吗？
- 3 今日板块顺序：① 自查周验收表 ② 同伴交叉验证 ③ 缺项如实标记
- 4 主任务：完成周验收表 + 同伴交叉验证
- 5 今日不做：新功能；顺手修复与验收无关的问题（记录到下周）
- 6 完成标准：周验收表逐项有证据；同伴给出三行结论的交叉验证；缺项如实标 FAIL；演示提纲已写出并完整走通一遍
- 7 今天交什么截图：三张——周验收表填完 + 同伴交叉验证的三行结论 + 演示提纲；图里要有逐项证据链接、同伴给出的可打开 / 可读写 / 无报错结论，以及提纲的四段结构（用户问题 → 核心流程 → 提示词改写 → 验证方式）
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 21 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：录一段 3 分钟演示视频
- 10 卡住降级：优先补齐「公网可访问 + 真实读写」两项证据，其余如实标记未执行

先讲清楚

第 3 周的验收标准不是「我写完了」，而是「证据拿得出来」：数据库真的有表和数据、接口真的能公网读写、检查台真的能发给别人打开。三项都有证据，这一周才算完成。同伴交叉验证只填三行结论——能否打开、能否真实读写、有无报错——零技术成本，但必须真做。

实战案例：今日热搜

演示：逐项过验收表——[建表脚本可重复执行](#) → PASS；[GET 公网返回真实数据](#) → PASS；[POST 写入并读回](#) → PASS；[检查台公网可访问](#) → PASS。最后按提纲做一次 3 分钟演示。

提示词怎么改

让 AI 根据项目现状[生成验收表](#)，每项必须写“怎么验证”；输出只允许 PASS、FAIL、未执行三种状态并附证据，不允许“基本完成”这类模糊表述。演示提纲的顺序固定：用户问题 → 核心流程 → 提示词改写 → 验证方式。

提示词参考：今日热搜

代码块

- 1 请根据「今日热搜」案例的当前完成情况，生成第 3 周验收表。
- 2
- 3 验收项至少包括：
- 4 - 建表和种子脚本可重复执行（有 select 证据）；
- 5 - GET 接口公网可访问且返回真实数据库数据；
- 6 - POST 接口完成真实写入并读回；
- 7 - 数据访问层重构完成且接口行为不变；

- 8 - 检查台公网可访问，同伴从自己设备打开成功；
- 9 - 响应形状与 `api-contract.md` 一致。
- 10
- 11 每项必须写明"怎么验证"；输出只允许 PASS、FAIL、未执行三种状态并附证据链接，不允许"基本完成"。
- 12 再生成一份 3-5 分钟演示提纲：用户问题 → 核心流程 → 提示词改写 → 验证方式。

换成自己的项目

代码块

- 1 请根据项目【项目名称】的当前状态生成第 3 周验收表和演示提纲。
- 2
- 3 验收项对应我本周产出：schema/seed 脚本、GET/POST 公网接口、分层重构、公网检查台 URL、`api-contract.md` 完整性。
- 4 每项写明验证方法，输出 PASS / FAIL / 未执行并附证据，不许用模糊表述。
- 5 演示提纲 3-5 分钟：用户问题、核心流程（含一次真实写入 + 刷新持久化）、提示词改写过程、验证方式、本周末完成项。
- 6 另给一份同伴交叉验证的三行结论模板：能否打开 / 能否真实读写 / 有无报错。

实战步骤

1. 让 AI 生成本周验收表（每项含验证方法）。
2. 逐项执行验证，记录 PASS / FAIL / 未执行 + 证据。
3. 完成 1 次同伴交叉验证，对方用你的公网链接独立验证并填三行结论。
4. 写 3-5 分钟演示提纲，并完整走通一遍。

完成后检查

- 验收表每项都有状态和证据。
- 同伴交叉验证的三行结论已填写。
- 演示提纲完整走通过一遍。
- FAIL 项都有说明和补救计划。

今天怎么检测

1. 验收表反假检测：让 AI 随机抽验收表里的 2 项，输入：「这两项声称 PASS，请告诉我对应的证据是什么，我现在重新执行一遍验证。」现场重做能对上 = 证据真实；对不上 = 该项改回 FAIL。验收表不允许有「点了但没真验」的项。

2. **交叉验证检测**：同伴用自己的设备（不是你的电脑）独立完成三步——打开你的公网链接、完成一次真实写入、观察有无报错——填回三行结论。三行都有明确结果 = 通过；同伴说「打不开」就是今天最高优先级问题。
3. **演示计时检测**：按提纲完整讲一遍并计时——5 分钟内走完核心流程、含一次真实写入 + 刷新持久化 = 合格；超时或漏掉核心流程 = 提纲要改。
4. **周盘点检测**：数一数本周证据清单——[Day15](#) 公网健康截图、[Day16](#) select 截图、[Day17](#) 真库对比截图、[Day18](#) 写入闭环截图、[Day19](#) 回归截图、[Day20](#) 公网页面截图——6 天的证据都找得到 = 本周成果可回放。

自检留档（群内只交一张截图）：验收表（含抽查结果）、同伴三行结论、演示计时记录、周证据清单。

每日一问

这周最花时间的是哪一步？你觉得值吗？

今天同时是周验证日（第 3 周）：除了下面这一问和一张截图，还要按 [附录 A](#) 的第三张卡，多交一份完整材料。

第 4 周 | 测试、部署和线上验收

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

注意：第 3 周你已经把作品部署到公网，本周不是「第一次上线」，而是把它升级为 v1.0（含发布、回滚与备份的最小生产实践）。本周主线：[Day 22](#) 补齐修改和删除 → [Day 23](#) 错误处理与安全边界 → [Day 24](#) 修一个真实 Bug → [Day 25](#) 制作发布前检查 Skill → [Day 26](#) 正式发布 v1.0 + 回滚演练 → [Day 27](#) 第三方试用和资源检查 → [Day 28](#) 正式验收。

本周产出：正式上线的 v1.0、一次真实回滚演练记录、一份可复用的发布前检查 Skill，以及完整的最终验收和演示。

本周 Day 目录

[Day 22](#) · [Day 23](#) · [Day 24](#) · [Day 25](#) · [Day 26](#) · [Day 27](#) · [Day 28](#)

Day 22 | 补齐修改和删除：PATCH / DELETE

今天用时：45–60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

补上修改（PATCH）和删除（DELETE）两类接口，让数据从「只能加」变成「能改能删」，数据操作闭环完整。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 22 | 第 4 周
- 2 今天要掌握：删除为什么比新增更容易出事？你在哪加了确认？
- 3 今日板块顺序：① PATCH 实现 ② DELETE 实现 ③ 四类操作闭环验证
- 4 主任务：PATCH / DELETE 上线并验证，数据操作闭环完整
- 5 今日不做：批量操作；用户系统
- 6 完成标准：PATCH 修改一条数据生效；DELETE 删除后 GET 不再返回；增删改查四类操作闭环
- 7 今天交什么截图：两张—PATCH 后的数据变化 + DELETE 后该条不再返回；图里要有改之前与之后的值对比，以及 GET 返回里该条已消失
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 22 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：给删除加软删除—不真删数据，只在记录上加一个 is_deleted 标记、查询时跳过，删错了还能找回
- 10 卡住降级：先保证一个接口（PATCH）完整可用，DELETE 记为待办

先讲清楚

前三周打通了「读和写」，但真实应用一定有改错、反悔的时候。今天补 PATCH（改一条）和 DELETE（删一条）。这天的另一个重点是「防呆」：删除是不可逆操作，前端要有二次确认；操作不存在的 id，接口要返回明确错误，不能崩溃。

实战案例：今日热搜

演示：写 PATCH `/api/favorites/:id` 修改收藏备注、DELETE `/api/favorites/:id` 取消收藏；每一步操作后到数据库 select 确认；再测一遍「删除一个不存在的 id」，看接口返回什么。

提示词怎么改

提示词里写明：按 id 操作、操作不存在的 id 返回什么、删除的二次确认由前端负责。最后要求 AI 同步更新 `api-contract.md` ——契约是活的文档，接口变了契约跟着变。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例补齐修改和删除接口，并给出部署与测试步骤。
- 2
- 3 要求：
- 4 - PATCH `/api/favorites/:id`：只允许修改备注字段；

- 5 - DELETE /api/favorites/:id: 删除一条收藏;
- 6 - 操作不存在的 id 时返回明确的中文错误, 不能崩溃或返回 500 裸报错;
- 7 - 响应统一 { ok, data, error };
- 8 - 前端在检查台页面上为删除操作增加二次确认;
- 9 - 给出每种操作的测试命令和数据库 select 前后对比验证方法;
- 10 - 最后把这两个接口补进 api-contract.md。

换成自己的项目

代码块

- 1 请为项目【项目名称】补齐修改和删除接口。
- 2
- 3 要求:
- 4 - PATCH 修改 checkins 的一条记录【可改字段按契约, 如完成状态、备注】;
- 5 - DELETE 删除一条打卡记录;
- 6 - 校验 id 存在性, 不存在的 id 返回中文错误说明;
- 7 - 前端删除操作加二次确认;
- 8 - 给出数据库 select 前后对比的验证方法;
- 9 - 同步更新 api-contract.md, 把两个新接口登记进去并标记"已实现"。

实战步骤

1. 让 AI 生成支持路径参数 :id 的 PATCH 和 DELETE 云函数（每类数据各 1 个，函数内通过参数中的 id 定位记录）。
2. 部署后在检查台页面上完成一次真实的修改和删除。
3. 数据库 select 确认修改、删除真实生效。
4. 更新 `api-contract.md`，登记这两个接口。

完成后检查

- PATCH 修改成功，数据库字段真实变化。
- DELETE 删除成功，数据不再出现在 GET 结果里。
- 操作不存在的 id 返回明确错误，不是崩溃。
- `api-contract.md` 已同步更新。

今天怎么检测

1. **修改检测**：在页面上把一条记录的备注改成「测试-改」，刷新仍在；再在控制台 `select` 看同一行——字段值真实变了。页面 + 数据库两端都看到 = 改的是真数据。（前后两张截图）

2. **删除检测**：在页面上删一条记录 → GET 结果里它消失了；控制台 `select` 里也没有了。两端都消失 = 真删了。
3. **防呆检测**：把 id 改成一个不存在的值（如 99999）调 PATCH 和 DELETE → 都应返回明确中文错误，页面不崩、接口不返回 500 裸报错。删一个不存在的数据还报「成功」= 防呆缺失，必须修。
4. **契约同步检测**：打开 `api-contract.md` → 能看到 PATCH / DELETE 两个新接口且标记"已实现"。契约没更新 = 接口变了文档没跟，验收日（Day 28）会踩自己挖的坑。

自检留档（群内只交一张截图）：修改前后对比截图、删除前后 select 对比截图、不存在 id 的错误响应截图、更新后的契约链接。

每日一问

删除为什么比新增更容易出事？你在哪加了确认？

Day 23 | 错误处理与安全边界

今天用时：45-60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

系统地处理三类错误（用户输入错、网络/接口错、服务端错），把密钥和配置移出代码，产出一份安全自查清单。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 23 | 第 4 周
- 2 今天要掌握：你把哪句裸报错改成了人话？改前改后分别是什么？
- 3 今日板块顺序：① 密钥排查（红线优先） ② .env 与忽略规则检查 ③ 三类错误提示统一
- 4 主任务：完成密钥排查 + 三类错误提示统一
- 5 今日不做：加复杂异常处理框架；改业务逻辑
- 6 完成标准：全仓库搜不到密钥特征词；.env 不在仓库且被忽略；三类错误都返回中文提示
- 7 今天交什么截图：两张——全仓库搜索密钥特征词的结果 + 三类错误的中文提示；图里要有搜索结果 0 条
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 23 | 一句话；说明两行：改了啥 / 加了啥），然后停下等我确认
- 9 余力加练：加一条简单的请求日志（时间、路径、结果）；可选：回补 Day 22 降级未完成的 DELETE 接口
- 10 卡住降级：优先做密钥排查（这是上线红线），错误提示可以明天补

先讲清楚

能跑的代码和能上线的代码，差的就是这一层。今天做三件事：一是把三类错误的提示统一成人能看懂的话；二是排查密钥——数据库连接串、API Key 绝不能出现在代码和 Git 里，统一走环境变量，仓库里只放不含真实值的 `.env.example`；三是明确边界：哪些输入接受、哪些拒绝。密钥排查是上线红线，优先级最高。

实战案例：今日热搜

演示：把「查询失败」的裸报错改成「数据暂时拿不到，请稍后再试」；把数据库连接串从代码挪进环境变量；在 Git 历史里搜一遍，确认没有密钥被提交过。

提示词怎么改

让 AI 先「审计」再修复：先列出项目里所有硬编码的密钥、所有会裸报错的地方，再逐条修复。修复结果必须输出一份安全自查清单（每项写"怎么算通过"），不允许只说"已修复"。

提示词参考：今日热搜

代码块

- 1
- 请对「今日热搜」案例做一次错误处理和安全审计，先列出问题再修复。
- 2
-
- 3
- 第一步审计，逐项列出：
- 4
- 所有硬编码的密钥、连接串、API Key；
- 5
- 所有未处理的错误路径（会把报错直接甩给用户的地方）；
- 6
- 所有可能的非法输入（空值、超长、类型错误、SQL 注入风险）。
- 7
-
- 8
- 第二步修复：
- 9
- 密钥全部移入环境变量，仓库提供 `.env.example`（不含真实值），确认 `.gitignore` 覆盖 `.env`；
- 10
- 错误分三类给统一中文提示：用户输入错（告诉他改什么）、网络/接口错（提示稍后再试）、服务端错（记录日志 + 通用提示）；
- 11
- 输出一份安全自查清单，每项写明"怎么算通过"，不许只写"已修复"。

换成自己的项目

代码块

- 1
- 请对项目【项目名称】做错误处理与安全审计，先审计后修复。
- 2
-
- 3
- 审计范围：硬编码密钥、裸报错、非法输入、`.gitignore` 完整性。
- 4
- 修复要求：
- 5
- 密钥走环境变量 + `.env.example`；
- 6
- 三类错误（输入错 / 网络错 / 服务端错）统一中文提示；

- 7 - 输出安全自查清单（每项含验证方法）；
- 8 - 在 Git 历史里搜索是否曾提交过密钥；一旦发现真实密钥，立即作废并重新生成、更新环境变量，不要只清理代码。

实战步骤

1. 让 AI 审计并列出现：硬编码密钥、裸报错、未处理的边界情况。
2. 逐项修复：错误分类提示、环境变量化、`.env.example`。
3. 检查 `.gitignore`，确认 `.env` 不会被提交。
4. 生成并保存安全自查清单。

完成后检查

- 三类错误都有人能看懂的提示。
- 代码和 Git 里搜不到任何密钥。
- 仓库里有 `.env.example`，没有 `.env`。
- 安全自查清单已保存。

今天怎么检测

1. **密钥搜查检测（红线，先做）**：把这句话发给 AI，让它代查并截图存证：「请对当前仓库执行两件事：① 全局搜索 password、secret、sk-、postgres:// 等特征词；② 检索全部 Git 提交历史中同样的特征词。列出所有命中的文件、提交和上下文，没有命中就明确说『0 命中』。」→ 历史提交无密钥泄漏。一旦发现真实密钥：立即到对应平台控制台作废并重新生成、更新环境变量——这一步优先于一切，只改代码或只清理历史都不算处理。发现任何一处 = 今天不许做别的，先处理。
2. **环境变量检测**：仓库里存在 `.env.example`（打开只有字段名没有真实值）、不存在 `.env`；执行 `git check-ignore .env` 原样输出 `.env = 已被忽略、提交不出去`；没有任何输出 = 没被忽略，需要修 `.gitignore`。
3. **三类错误实测检测**：亲手各触发一次——①提交一个格式错误的输入（用户输入错）②把后端停掉或断网再操作（网络错）③临时改错表名（服务端错）——三次都应看到对应的中文提示，而不是英文堆栈或白屏；页面提示不明显的，按 F12 在 Console/Network 面板里看报错（用法见[附录 K](#)）。测完改回来。（三张截图）
4. **清单可执行检测**：把安全自查清单交给同伴，同伴只看清单就能逐项执行并得出同样结论 = 清单写得够清楚；同伴看不懂哪项 = 那项重写。

自检留档（群内只交一张截图）：密钥搜索 0 命中截图、三类错误实测截图、`.env.example` 文件、自查清单链接。

每日一问

你把哪句裸报错改成了人话？改前改后分别是什么？

Day 24 | 用一个真实 Bug 练习定位和修复

今天用时：45-60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

不写新功能，专门用一份测试清单找 Bug、定位 Bug、修复 Bug，留下「修复前」和「修复后」两份证据。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 24 | 第 4 周
- 2 今天要掌握：你是怎么排除掉其他原因、锁定真正原因的？
- 3 今日板块顺序：① 复现 Bug ② 定位 ③ 修复 ④ 验证留证
- 4 主任务：修复一个真实 Bug 并留全证据
- 5 今日不做：顺手修别的 Bug（记录下来）
- 6 完成标准：复现、定位、修复、验证四步证据齐全；修复已提交
- 7 今天交什么截图：两张—F12 控制台里的报错原文 + 修复后正常运行；图里要有关键错误原文那几行，以及地址栏或正常返回
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 24 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：修复第二个低优先级 Bug
- 10 卡住降级：找不到 Bug 就让 AI 用空值、超长文本、快速连点等刁钻输入生成补充测试用例

先讲清楚

修 Bug 的能力比写新功能更能看出水平。今天的练习方法固定为四步：按清单测试 → 选一个最影响核心流程的 Bug → 按模板让 AI 定位 → 修复后用同一清单回归验证。给 AI 的信息只有四样：现象、复现步骤、报错原文、已尝试动作——不要替 AI 下结论（「我觉得是 XX 问题」不要写），让证据说话。

实战案例：今日热搜

演示：按测试清单逐项操作，发现「快速连点收藏会写入两条数据」；把现象和复现步骤给 AI 定位；修复（防抖：快速连点时只认第一次点击；或数据库唯一约束）后按同一清单回归，确认修复生效且没引入新问题。

提示词怎么改

修 Bug 提示词的要点：只给现象和证据；必须包含复现步骤、报错原文、已尝试动作；要求 AI 先给原因排序再给验证方法，最后附回归验证清单。找不到 Bug 时，让 AI 用更刁钻的输入（空值、超长文本、快速连点）生成补充测试用例。

提示词参考：今日热搜

代码块

- 1 请为「今日热搜」案例生成一份覆盖核心流程的测试清单，用于今天集中找 Bug。
- 2
- 3 清单覆盖：打开首页、读取列表、收藏写入、刷新持久化、修改备注、取消收藏、重复操作、空值和超长输入、快速连点。
- 4
- 5 我执行后会把发现的问题按下面格式反馈给你，请只基于我给的信息定位：
- 6 【现象】.....【复现步骤】.....【报错原文】.....【已尝试动作】.....
- 7
- 8 定位时：先给最可能的原因排序，再给每个原因的验证方法；修复方案给出后，同时给一份回归验证清单，确认修复生效且没有引入新问题。不要在没看到证据前直接下结论。

换成自己的项目

代码块

- 1 请为项目【项目名称】生成核心流程测试清单（完整链路：打开检查台 → 读取 plan_days/checkins → 写入打卡 → 刷新确认 → 修改 → 删除）。
- 2
- 3 我会按【现象 / 复现步骤 / 报错原文 / 已尝试动作】格式反馈问题。
- 4 请按顺序：原因排序 → 每个原因的验证方法 → 修复方案 → 回归验证清单。
- 5 修复后我要用同一清单重新跑一遍，前后各留一份证据。

实战步骤

1. 让 AI 生成一份覆盖核心流程的测试清单。
2. 逐项执行，记录所有异常现象。
3. 选最高优先级的一个 Bug，按模板让 AI 定位。
4. 修复后用同一清单回归验证，留修复前后两份证据。

完成后检查

- 至少一个 Bug 完整走完「发现 → 定位 → 修复 → 回归」。
- 修复前后证据都已保存。
- 回归验证确认没有引入新问题。

· 其余未修复 Bug 都有记录和优先级。

今天怎么检测

- 1. **可复现检测**：修复前，那个 Bug 按你写的复现步骤**连续执行 3 次，3 次都能复现** = 复现步骤可靠；3 次里只出现 1 次 = 步骤不完整，先补步骤再谈修复。
- 2. **修复生效检测**：修复后，用**完全相同的复现步骤**再执行 3 次 → 3 次都不再出现 = 修好了。修复前 3 次「坏」、修复后 3 次「好」，前后各留录屏或截图。
- 3. **无副作用检测（回归）**：完整跑一遍修复前的测试清单 → 除已修复项外，其他项结果和修复前一致。修好一个、弄坏两个 = 不合格，回滚重修。
- 4. **定位能力自检**：合上 AI 给的分析，自己回答：「这个 Bug 的根本原因是什么？AI 的哪条线索排除了其他原因？」 答不出 = 只是复制了修复代码，没有消化，记入「待复盘」。

自检留档（群内只交一张截图）：修复前复现录屏/截图、修复后 3 次验证记录、回归清单结果、根本原因的一段话。

每日一问

你是怎么排除掉其他原因、锁定真正原因的？

Day 25 | 做一个「发布前检查」 Skill

今天用时：45-60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

把前两周积累的检查经验，固化成一个可复用的 Skill：以后每次发布前，一句话让 AI 自动过一遍所有检查项。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 25 | 第 4 周
- 2 今天要掌握：你故意弄坏哪一项来验证 Skill 不会虚报通过？
- 3 今日板块顺序：① 规划检查项 ② 制作检查 Skill ③ 实跑验证
- 4 主任务：产出可复用的发布前检查 Skill
- 5 今日不做：把检查 Skill 做成自动化流水线
- 6 完成标准：Skill 文件已入库；对当前项目实跑一遍并给出结果；故意留个问题它能查出来
- 7 今天交什么截图：发布前检查 Skill 跑出来的结果——图里要能看出你故意留的问题被它查出来了

- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 25 | 一句话；说明两行：改了什麼 / 加了什麼），然后停下等我确认
- 9 余力加练：把 Skill 的用法写进项目 README
- 10 卡住降级：先做最小版本（只查公网 + 密钥两项），以后逐步加

先讲清楚

前面的检查靠人记：密钥有没有漏、接口通不通、数据库能不能写、跨域处理没处理……人会忘，Skill 不会。今天把这些检查项写成一个项目 Skill（verify-project）：以后每次改完代码、每次发布前，都能一键复用。

实战案例：今日热搜

演示：创建 verify-project Skill，检查项覆盖：公网地址可访问、健康接口正常、读写接口正常、无硬编码密钥、`.gitignore` 完整、数据库可连接。在项目里真实运行一次，输出逐项 PASS / FAIL 报告。

提示词怎么改

让 AI 生成 Skill 时，检查项要来自「这个项目真实踩过的坑」，而不是通用模板。每个检查项必须写清“怎么算通过”，禁止“看起来正常”这类模糊表述。生成后至少运行两次：第一次找误报和漏报，修正后再跑第二次。

提示词参考：今日热搜

代码块

- 1 请帮我创建一个名为 verify-project 的项目 Skill，用于每次发布前自动检查。
- 2
- 3 检查项必须来自本项目真实踩过的坑，至少包括：
- 4 - 公网首页可访问且展示数据库真实数据；
- 5 - 健康接口 GET /api/health 正常；
- 6 - 读写接口各调用一次且结果正确；
- 7 - 代码和 Git 里搜索不到硬编码密钥；
- 8 - .gitignore 覆盖 .env，仓库存在 .env.example；
- 9 - 数据库可连接，核心表存在。
- 10
- 11 要求：
- 12 - 每个检查项写明执行命令和“怎么算通过”；
- 13 - 输出格式为逐项 PASS / FAIL + 证据；
- 14 - 不允许“看起来正常”“基本没问题”这类模糊表述；
- 15 - 给出 Skill 的文件结构和在对话中调用它的方法。

换成自己的项目

代码块

- 1 请为项目【项目名称】创建 verify-project Skill。
- 2
- 3 检查项从我项目的踩坑记录里整理：【列出本项目真实遇到的问题，如跨域报错、环境变量未生效、重复写入等】。
- 4 每项写明执行命令、通过标准，输出逐项 PASS/FAIL + 证据。
- 5 生成后先运行一次，把误报和漏报的检查项反馈给你修正，再运行第二次确认。

实战步骤

1. 整理本项目所有踩过的坑和对应检查项。
2. 让 AI 生成 verify-project Skill 文件。
3. 真实运行一次，逐项核对输出是否可信。
4. 修正误报、漏报的检查项，再运行一次确认。

完成后检查

- Skill 存在且真实运行过。
- 覆盖：公网、健康、读写、密钥、Git 安全、数据库。
- 输出为逐项 PASS / FAIL + 证据，没有模糊表述。
- 至少运行过两次（第二次修正过误报或漏报）。

今天怎么检测

1. **真实工作检测（防止 Skill 走形式）**：故意弄坏一项——比如临时把某个接口路径改错、或在代码里塞一个假密钥——再运行 verify-project。对应项必须变 FAIL = Skill 真的在检查；仍然全 PASS = Skill 是摆设，检查项没落到实处。测完改回来。（坏→FAIL、好→PASS 两张截图）
2. **输出可信检测**：把 Skill 的 PASS 项随机抽 2 项，自己亲手按它的"怎么算通过"重做一遍 → 结论一致 = 可信；不一致 = 该项的标准写得太虚，回炉重写。
3. **复用性检测**：新开一个对话，只说「运行 verify-project」→ AI 能正确加载并执行 = 复用成立；新对话里它不知道怎么跑 = Skill 内容缺上下文，补自足说明。（如果你的 AI 工具不能保存 Skill 文件：把 Skill 内容存为仓库里的 SKILL.md，新开对话时先粘贴全文，再说“按这份清单逐项检查”。）
4. **覆盖度检测**：翻一遍 Day15-24 的「完成后检查」，把当时踩过的坑列出来对照 Skill 检查项——每条坑都有对应检查 = 覆盖完整；有漏 = 补上。

自检留档（群内只交一张截图）：故意弄坏再检测的两张对比截图、抽查验证记录、新对话复用截图。

每日一问

你故意弄坏哪一项来验证 Skill 不会虚报通过？

Day 26 | 正式发布 v1.0 和回滚演练

今天用时：90 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

把第 3 周的检查台正式固化为 v1.0 发布，并真实执行一次回滚演练——亲手做静态托管回退这一条路径，云函数回退和数据恢复写进手册备查。既知道怎么上线，也知道出事怎么退回去。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 26 | 第 4 周
- 2 今天要掌握：回滚演练花了多久？真出事你敢不敢用这套流程？
- 3 今日板块顺序：① 确认可回退历史 ② 发布 v1.0 ③ 静态托管回退演练 ④ 恢复 v1.0
- 4 主任务：正式发布 v1.0 + 完成一次真实回滚演练（静态托管回退路径）
- 5 今日不做：云函数回退和数据恢复的实操（只写进回滚手册备查）
- 6 完成标准：v1.0 公网可访问且检查全过；回滚演练有耗时记录；恢复后公网正常
- 7 今天交什么截图：三张——v1.0 公网可访问 + 回滚前后的提交记录 + 回滚后页面仍正常；图里要有地址栏和页面
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 26 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：给回滚手册加一张「什么情况必须回滚」的判断表
- 10 卡住降级：优先完成发布；回滚演练受阻时至少完成「退回上一版」这一步并记录

先讲清楚

第 15 天你已经打通过部署，今天不是「第一次上线」，而是「正式固化」：打上 v1.0 版本标记，记录发布清单。同时做一次回滚演练：假设刚发布的版本有问题，如何在短时间内退回上一版。没演练过的回滚等于没有回滚——真出事的时候你会手忙脚乱。所以今天必须真的执行一遍，不允许只"列出方法"。另外分清两件事：回滚只解决代码问题——退回上一版部署；数据一旦被误删，回滚救不回来，只能靠第 27 天导出的备份重新导入。备份和回滚要分开练。

实战案例：今日热搜

演示：用上一个稳定版本覆盖当前版本（模拟新版本出问题），确认公网恢复可用、数据无损坏；记录整个回滚用了几分钟、每一步做了什么，形成回滚手册；最后恢复 v1.0，再次验证。

提示词怎么改

让 AI 生成两份文档：发布清单（版本号、发布内容、验证项、备份方式）和回滚手册（触发条件、分步操作、验证方法）。回滚演练必须真做，并记录实际耗时。

提示词参考：今日热搜

代码块

- 1
- 请为「今日热搜」案例生成两份文档，并指导我完成一次真实回滚演练。
- 2
- 3
- 一、发布清单（v1.0）：
- 4
- 版本号、发布内容说明；
- 5
- 发布前必须通过的验证项（可直接引用 verify-project Skill 的输出）；
- 6
- 发布方式和发布后逐项验证步骤；
- 7
- 备份方式（代码版本标记 + 数据导出）。
- 8
- 9
- 二、回滚手册：
- 10
- 触发条件：什么情况必须回滚；
- 11
- 具体步骤分三层：代码回退（静态托管回退上一版）、云函数回退（重新部署上一版代码）、数据恢复（只能通过发布前导出的备份重新导入；发布含表结构变更前必须先导出数据）；
- 12
- 回滚后的验证方法。
- 13
- 14
- 要求：回滚演练必须真实执行一遍——把当前版本替换回上一稳定版本，确认公网恢复可用、数据无损坏，记录实际耗时，然后再恢复 v1.0。不允许只"列出方法"。

换成自己的项目

代码块

- 1
- 请为项目【项目名称】生成发布清单（v1.0）和回滚手册。
- 2
- 3
- 发布清单含：版本标记、发布前验证项、发布步骤、发布后验证、代码与数据备份方式。
- 4
- 回滚手册含：触发条件、静态托管与云函数各自的回退步骤、回滚后验证。
- 5
- 6
- 指导我真实执行一次回滚演练：退回上一版 → 验证公网可用 → 记录耗时 → 恢复 v1.0 → 再次验证。
- 7
- 输出演练记录模板：每一步做了什么、耗时多少、结果如何。

实战步骤

1. 先确认有可回退的历史：近 7 天有可用的历史提交或版本记录，没有则先补一次。然后让 AI 生成发布清单和回滚手册。
2. 按 v1.0 正式发布，逐项验证。
3. 真做一次回滚演练，范围是静态托管回退这一条路径：在控制台把静态托管退回上一版 → 验证公网可用 → 记录耗时 → 恢复 v1.0。云函数回退和数据恢复这两条路径只写进回滚手册，标注「手册内容，不实操」，并写一句不实操的原因。
4. 把演练记录和回滚手册存进项目仓库。

完成后检查

- v1.0 已发布，公网核心功能逐项验证通过。
- 回滚演练真实执行过，不是只写了方法。
- 回滚手册包含触发条件和分步操作。
- 发布前的备份和版本标记存在。

今天怎么检测

1. **发布成立检测**：v1.0 发布后运行一次 [verify-project Skill](#) → 全部 PASS；公网地址浏览器打开、核心流程走一遍 → 正常。发布是否成立，以 Skill 输出和亲手走流程为准，不以「我部署了」为准。
2. **回滚真实性检测**：演练记录里必须有三个硬数据——回滚前的时间、恢复公网可用的时间、总耗时几分钟。没有数字 = 只写了方法没真做。演练后公网两次验证都要过：退回后可用、恢复 v1.0 后可用。
3. **手册可执行检测**：把回滚手册交给同伴，对照手册逐条走查，请他指出他预判会卡住的步骤（你在旁边只观察不出手）；单人学员就自带手册慢速执行一遍并标注疑问。同伴预判会卡的步骤 = 那一步写得不够细。
4. **备份存在检测**：能指出版本标记在哪（git tag / 备份文件 / 云端快照——某一时刻数据的完整备份），并当场用它说出「上一版本是哪一版、怎么取回」。说不出 = 备份只是个念头，不是资产。

自检留档（群内只交一张截图）：verify-project 全 PASS 输出、含耗时的回滚演练记录、同伴手册执行记录、版本标记截图。

每日一问

回滚演练花了多久？真出事你敢不敢用这套流程？

Day 27 | 第三方真实用户试用和资源检查

今天用时：45-60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

请一位不懂代码的真实用户完成一次完整试用，从他的卡点里修最高优先级的一个问题；同时检查 CloudBase 资源用量和到期风险。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 27 | 第 4 周
- 2 今天要掌握：真实用户卡在哪个你从没想过的地方？
- 3 今日板块顺序：① 邀请第三方试用 ② 收集反馈 ③ 最高优先级修复 ④ 资源额度检查
- 4 主任务：完成第三方线上试用和一次最高优先级修复
- 5 今日不做：按反馈大改；加新功能
- 6 完成标准：一位没参与项目的真人完成核心流程；最高优先级问题已修复；资源额度已检查
- 7 今天交什么截图：两张—第三方试用的记录 + 资源检查结果；图里要有对方设备上完成核心流程的画面，以及额度、到期日期、用量
- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 27 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：用手机宽度完成一次完整打卡流程
- 10 卡住降级：自己间隔 10 分钟后按试用任务重新操作，记录第一个找不到或不理解的地方

先讲清楚

你和同伴测不出的问题，真实用户一测就现形——因为他不看代码、不读文档，只按直觉点。今天全程记录试用：哪一步找不到、哪句话看不懂、哪里点错了。只修一个最高优先级问题，不贪多。另外做一次资源体检：额度用了多少、什么时候到期、数据怎么导出——别让作品因为欠费悄悄消失。

实战案例：今日热搜

演示：请一位同学用手机完成「打开首页 → 读热搜 → 收藏 → 刷新确认 → 修改备注 → 取消收藏」，全程不提示；记录他卡住的 3 个地方；修复其中最影响流程的一个；最后导出一份不含密钥的数据备份。

提示词怎么改

试用任务要让 AI 生成「给外行的操作脚本」：每一步只说"点什么"，不说原理。修复提示词沿用 Day 24 的格式（现象 + 复现步骤 + 已尝试动作），修复后必须请用户复验一次。

提示词参考：今日热搜

代码块 请为「今日热搜」案例生成一份给外行用户的试用任务脚本。

- 2
- 3 要求：
- 4 - 用户全程只用手机、不看任何代码和文档；
- 5 - 每一步只说"点什么"，不说原理，例如：打开链接 → 找到一条热搜 → 收藏 → 刷新页面确认收藏还在 → 修改备注 → 取消收藏；
- 6 - 把 1-2 处文案写得略含糊（如按钮只写"提交"不写"提交打卡"），观察用户是否自己能理解；
- 7 - 附一份记录表模板：哪一步卡住、用户当时说了什么、属于找不到还是看不懂。
- 8
- 9 我拿到试用记录后，请帮我：从卡点里选出最高优先级的一个，按【现象 / 复现步骤 / 报错原文 / 已尝试动作】整理成修复提示词；修复后生成一个只验证修复点的最小复验任务。

换成自己的项目

代码块

- 1 请为项目【项目名称】生成外行试用任务脚本和记录表模板。
- 2
- 3 试用链路：打开公网链接 → 看到你应用的核心数据 → 完成一次核心操作（打卡应用示例：完成一次打卡）→ 刷新确认 → 修改 → 删除。
- 4 每步只说"点什么"，不说原理；把 1-2 处文案写得略含糊。
- 5 拿到记录后：帮我选最高优先级卡点，按修复模板整理提示词，修复后给用户复验的最小任务。
- 6
- 7 另外生成 CloudBase 资源检查任务：查看当前用量、到期时间、导出数据的操作步骤，输出一份"资源与到期说明"（不含任何密钥）。

实战步骤

1. 让 AI 生成给外行的试用任务脚本。
2. 请一名真实用户独立完成试用，全程记录卡点。
3. 修复最高优先级的一个问题，请用户复验。
4. 检查 CloudBase 资源用量、到期时间，导出一份数据备份。

完成后检查

- 有完整的外行试用记录，不是自己测自己。
- 最高优先级问题已修复并经用户复验。
- CloudBase 额度、到期时间、导出方式已成文。
- 备份不含密钥和隐私数据。

今天怎么检测

1. **试用真实性检测**：试用记录里要有用户的**原话**（哪怕只言片语）和他卡住时的操作路径，而不是你的总结转述。有原话 = 真试用；全是转述 = 可能是自己代填。记录里至少有 1 处你没预料到的卡点——一个都没有，多半是用户客气了，换人再测。
2. **修复复验检测**：修复后，请**同一位用户**只做「验证修复点」的最小任务 → 他独立完成且不再卡住 = 修复有效；你自己测着没问题但用户仍卡 = 没修到点上。
3. **资源可见检测**：CloudBase 控制台截图里能同时看到——当前用量、到期日期（或计费方式）。两个信息缺一个 = 体检没做完。
4. **备份可用检测**：打开导出的备份文件确认内容完整；在文件里搜索密钥特征词 → 0 命中 = 不含敏感信息；再答一句：「如果明天环境没了，用这份备份 + 仓库代码，多久能恢复？」有答案 = 备份是真保险。

自检留档（群内只交一张截图）：含用户原话的试用表、复验记录、控制台用量与到期截图、备份文件说明。

每日一问

真实用户卡在哪个你从没想过的地方？

Day 28 | 正式验收，交付一个能用、能解释、能继续改的作品

今天是结营日，材料比平时多，属正常。交付清单是 [附录 J](#) 里的 8 项，照着核对一遍就行。结营后你想把项目放到自己账号下当作品集，找管理员发起一次转移就行。

今天用时：45–60 分钟

[返回总目录](#) · [返回第 4 周](#) · [本周 Day 目录](#)

学习目标

完成最终验收和 3–5 分钟演示，交付公网链接、项目文档、验证证据和后续优化计划。

今日任务清单（复制发给 AI）

代码块

- 1 【今日任务清单】Day 28 | 第 4 周
- 2 今天要掌握：28 天里你最大的一个认知改变是什么？
- 3 今日板块顺序：① 验收官检测 ② 核心流程重复检测 ③ 交付清单清点 ④ 结营复盘
- 4 主任务：交付你自己的 28 天作品 v1.0（主题自选，打卡应用只是示范案例）
- 5 今日不做：任何影响最终验收的新功能
- 6 完成标准：附录 J 清单逐项能当场打开或展示；验收官检测通过；复盘写完
- 7 今天交什么截图：两张——附录 J 验收清单逐项打勾 + 作品公网 URL 打开的样子；图里要有清单的勾选状态，以及地址栏和完整作品

- 8 收尾指令：全部完成后，列出改动文件清单并说明各属于哪天的任务，提交并推送（commit 标题：Day 28 | 一句话；说明两行：改了什么 / 加了什么），然后停下等我确认
- 9 余力加练：增加数据导出按钮、简单完成率统计或部署日志提醒，但不能影响最终验收
- 10 卡住降级：先保证公网首页、健康接口、一次线上写入和刷新后持久化，其余内容标记为未完成并说明原因；若 PATCH/DELETE 有未完成项，如实标记未执行并写明原因，该项扣分但不阻塞结营，有余力在 Day 23–27 任一天回补

先讲清楚

结营不是把页面打开一次，而是证明核心流程可以重复完成。一个能继续改的作品，至少要有清楚的数据模型、可调用的 API、线上地址、真实数据库记录和失败时的说明。

实战案例：今日热搜

演示：打开线上首页，读取真实热搜，查看详情，完成收藏，刷新后确认收藏仍在，修改备注，再取消收藏。最后展示 API、数据库记录和部署验收表。

提示词怎么改

验收项必须可以观察；每项输出 PASS、FAIL 或未执行；演示要先说用户问题，再展示核心流程，最后说明提示词、验证方式和下一步。不要因为首页能打开就宣称全部完成。

提示词参考：今日热搜

代码块

- 1 请根据当前项目文档和公网地址生成最终验收表和 3–5 分钟演示提纲。
- 2
- 3 逐项检查：
- 4 - 至少 3 个页面或独立视图；
- 5 - 至少 1 个有明确反馈的交互；
- 6 - 至少 1 类数据真实写入并读取数据库；
- 7 - 前端、云函数和数据库完整链路；
- 8 - 公网首页、健康接口和线上持久化；
- 9 - CloudBase 资源和到期风险已记录；
- 10 - 回滚手册和数据导出方式已交付；
- 11 - 密钥、.env 和私人信息没有进入 Git。
- 12
- 13 每项输出 PASS、FAIL 或未执行并附证据，不要把未验证说成通过。
- 14 演示包含：用户问题、核心流程、提示词改写、验证方式、平台限制和后续优化计划。

换成自己的项目

代码块

- 1 请根据【项目文档和公网地址】生成最终验收表和 3-5 分钟演示提纲。
- 2
- 3 逐项检查：【页面/视图数量】【反馈交互】【数据库读写】【前后端链路】【合法数据来源】【公网地址】【线上持久化】【回滚与备份说明】【CloudBase 额度和到期风险】【敏感文件】。
- 4
- 5 每项输出 PASS、FAIL 或未执行并附实际证据，不要把未验证内容说成通过。
- 6 演示包含：【用户问题】【核心流程】【提示词改写】【验证方式】【平台限制】【后续优化计划】。

实战步骤

1. 运行最终验收表。
2. 处理仍然阻塞核心流程的问题。
3. 整理公网地址、API 文档（`api-contract.md`）、数据库说明和部署清单。
4. 按 3-5 分钟提纲演示一次。
5. 写下已完成、未完成和后续优化计划（例如：接一个真实数据源→附录 F、加注册登录→附录 D、给核心记录加统计页——复用现有 GET 接口加前端图表；课程到结营为止，这些是结营后自己迭代的优化方向）。

完成后检查

- 公网首页和健康接口都能访问。
- 至少一类数据完成了真实写入、读取和刷新持久化。
- 最终验收表为每一项提供了状态和证据。
- 部署限制、环境到期时间、回滚手册和备份方式写进交付说明。
- 3-5 分钟演示能完整走通核心流程。
- 未完成内容已经如实标记。

今天怎么检测

1. **验收官检测（今天的核心动作）**：让 AI 扮演「不认识你的验收官」，只给它公网地址和交付文档（不给你的解释），让它独立核验最终验收表并随机抽 3 项要求现场复验。全部对上 = 验收成立；有一项对不上 = 该项如实改回 FAIL 并说明。自己不能当自己的验收官。
2. **核心流程重复检测**：完整走两遍「写入 → 刷新 → 持久化 → 修改 → 删除」，两遍结果一致 = 可重复完成；只成功一遍 = 还不算能交付。
3. **交付清单清点检测**：对照附录 J 的结营交付清单逐项打勾，每项都能当场打开或展示 = 交付完整。
4. **「能继续改」检测**：让 AI（或同伴）只看仓库和文档，回答："如果要给你的核心记录加一个统计功能（打卡应用对应"统计完成率"），从哪张表、哪个接口开始改？"它能给出合理路径 = 架构和文档足够清楚，作品真的能继续改。

自检留档（群内只交一张截图）：验收官核验结果、两遍核心流程记录、8 项交付清单、可继续改的问答记录。

每日一问

28 天里你最大的一个认知改变是什么？

今天同时是周验证日（第 4 周，最终验收）：除每日一问和一张截图外，还要交最终验收材料——直接用[附录 J](#) 的 100 分验收表、结营交付清单 8 项和验收官检测结果，不再单独填周验收表。

附录

[返回总目录](#) · [返回第 1 周](#) · [返回第 2 周](#) · [返回第 3 周](#) · [返回第 4 周](#)

本部分共 13 篇，按四类组织：起步与规范、选型与云端操作、数据与扩展、排错与验收；用于改写提示词、理解云端术语、注册与部署 CloudBase、处理真实数据、排查错误并完成最终验收。

起步与规范：[附录 A | 打卡模板与求助模板](#) · [附录 B | 通用提示词五件套与 AI 操作边界](#) · [附录 L | 项目文件命名规范](#)

选型与云端操作：[附录 C | 零基础云端术语表](#) · [附录 D | 工具、技术栈和账号路线选择](#) · [附录 E | CloudBase PostgreSQL 初始化、备份与持久化](#) · [附录 H | CloudBase 部署、额度和到期检查](#) · [附录 M | CloudBase 注册、控制台与使用指导手册](#)

数据与扩展：[附录 F | 真实数据来源、替换和合规](#) · [附录 G | Skill 创建与验收](#)

排错与验收：[附录 I | 常见问题快速处理](#) · [附录 K | 浏览器开发者工具（F12）速查](#) · [附录 J | 100 分最终验收与项目映射表](#)

附录 A | 打卡模板与求助模板

[返回总目录](#) · [回到 Day 1](#)

一、学员每日打卡（1 问 + 1 张成果截图）

代码块

- 1 Day 15 | 每日一问
- 2 答：<一句话，说清今天做了什么、你的判断是什么>
- 3 图：<当天要求的成果证据，具体截什么见当天任务清单的「今天交什么截图」>

三条说明：① 问题每天不一样，在当天任务清单之后的「每日一问」小节里。② 答不出来、或者只能写「完成了」，就写清卡在哪一步，这一步也算答。③ 截图截的是今天做出来的东西（页面、接口、数据库、文件、提交记录），不是「每日一问」的答案，也不是群聊截图；每天截什么见当天任务清单的「今天交什么截图」。

二、commit 标题与说明（每天提交时用）

- 1 标题：Day 15 | 打通云端链路
- 2 说明：
- 3 改了什么：<哪个文件，改成了什么>
- 4 加了什么：<新增的文件或功能>（没有的写「无」）

标题里的 Day 15 是当天编号，28 个提交连起来就是你的进度条。说明写客观的文件变更；「每日一问」写你自己的判断，两者不重复。

三、周验证日模板（Day 7 / 14 / 21 / 28）

代码块

- 1 第 X 周 | 周验证日
- 2 姓名 / 校区 / 项目名称：
- 3 本周完成的主要任务：
- 4 本周检测执行结果：
- 5 证据链接（截图 / 录屏 / 公网 URL；页面 / API / 云函数 / 数据库）：
- 6 完成标准：已完成 / 部分完成 / 未完成
- 7 同伴交叉验证：可打开 ☐ 可真实读写 ☐ 无报错 ☐
- 8 遇到的问题 + 报错原文与已尝试动作：

日常那 24 天只交「1 问 + 1 图」，这 4 天多交一份上面的完整材料。第 28 天的周材料直接用[附录 J](#)的最终验收表和结营交付清单，不再单独填本模板。周验证日是把这一周的东西摊开看一遍，材料多是正常的。

提交前先隐藏密码、Cookie、Token、数据库连接串、环境变量值与私人路径。

四、求助模板（卡住了才用，不是每日项）

代码块

- 1 【Day / 项目名称】
- 2 【当前步骤】Step X
- 3 【我想达到的结果】
- 4 【实际看到的结果】
- 5 【第一条关键错误原文】
- 6 【我已经尝试过】
- 7 【相关页面、API、云函数日志或数据库证据】
- 8 【设备、浏览器和公网地址】
- 9 【我希望大家帮我判断的一个问题】

先隐藏密码、Cookie、Token、数据库连接串、环境变量值和私人路径，再提交问题。

附录 B | 通用提示词五件套与 AI 操作边界

通用提示词骨架

代码块

- 1
- 背景：我正在制作【项目】，给【用户】解决【问题】。
- 2
- 当前路线：前端【React/Vite 或实际方案】；后端 CloudBase Node.js 云函数；数据库 CloudBase PostgreSQL；前端部署到 CloudBase 静态网站托管。
- 3
- 4
- 目标：今天只完成【一个可观察成果】。
- 5
- 具体内容：必须包含【页面/接口/数据/动作/字段】。
- 6
- 约束：保持【不变内容】；暂时不做【功能】；不要输出或要求我粘贴真实密钥；不要开启付费套餐、按量计费或预置并发；不要凭空编造平台命令。
- 7
- 8
- 执行前：先读取【PRD/TECH_DESIGN/package.json/真实配置】，说明理解、假设、修改文件和验证方法。
- 9
- 执行中：每次只完成一个小步骤；遇到登录、授权、密钥、计费、删除云资源或覆盖数据时先停下来。
- 10
- 执行后：报告修改文件、运行命令、实际结果，并按 PASS、FAIL 或未执行给出证据。
- 11
- 验证：告诉我如何通过【页面/请求/云函数日志/数据库查询/公网地址】确认完成。

AI 可以直接做与必须停下来的事

AI 可直接执行	需要学员确认后执行	禁止直接执行
读项目文件、写代码、安装项目依赖、运行测试、生成部署清单	登录 CloudBase、授权 CLI、创建环境、填写环境变量、发布云函数和静态网站	粘贴真实密钥、开启付费/按量计费、删除环境或数据库、覆盖正式数据、把 CORS 配置成允许所有来源（*）
创建 .env.example、迁移脚本和验证脚本	执行会改变云端数据的初始化/迁移命令	绕过验证码、反爬、登录限制或使用他人 Cookie/Token

附录 C | 零基础云端术语表

术语	白话解释	在本训练营中的例子
前端	用户看到和操作的页面	React/Vite 页面、按钮、表单
后端	处理请求和业务规则的程序	CloudBase Node.js 云函数
API	前端调用后端的入口	GET /api/health

云函数	收到请求时运行的一段后端代码	读取计划或写入打卡
HTTP 访问服务	把公网请求转给云函数	把 /api 路径交给后端
PostgreSQL	保存结构化数据的数据库	plan_days 和 checkins 表（示例表名，请按你的主题替换）
表/字段/主键	数据的分类、属性和唯一编号	checkins.id、plan_day_id
唯一约束	阻止不应重复的数据	同一天同一计划项只打卡一次（打卡应用示例）
参数化查询	把数据与 SQL 指令分开	避免把用户输入拼进 SQL
环境变量	部署时填写、代码中不写死的配置	数据库连接信息和环境 ID
静态网站托管	把构建后的前端文件放到公网	CloudBase 静态网站托管
持久化	刷新或重新运行后数据仍存在	打卡写入 PostgreSQL 后仍可读取
冷启动	服务一段时间没用后首次访问较慢	云函数第一次请求等待
资源额度	免费环境可使用资源的上限	调用次数、流量、存储等
回滚	发布失败后恢复上一版	重新部署上一个可用版本
数据导出	把云端数据保存为备份	导出表结构和必要记录
跨域 / CORS	浏览器的一道安全门，规定「A 网站的页面不能随便去 B 网站的接口取数据」。没配好时页面能打开，但列表是空的	前端在静态托管域名，去调云函数的 /api 接口，需在控制台把两边设为互相信任
mock 数据	手写的假数据，先把页面撑起来看效果，等接口好了再换成真数据	Day 8 的热搜列表先用本地假数组渲染
提交 / 推送	提交是把改动存成一个版本记录；推送是把这个记录发到 GitHub	Day 2 的第一次 commit 和 push
构建 / 构建产物	把源代码加工成浏览器能直接跑的文件，产物通常在 dist 目录	npm run build 之后得到的 dist 文件夹
MVP	最小可行产品，用最少的功能验证核心价值，先跑通再补	Day 7 首页能打开、能显示列表就够了
Skill	给 AI 的一套工作说明，写清检查什么、遵守什么、怎么验证	附录 G 的 verify-project
AGENTS.md	项目规则文件，WorkBuddy 每次开工都会先读它	Day 1 创建，Day 6 追加个人规则

HTTP 状态码	服务器回应的编号：200 成功、4xx 请求有问题、5xx 服务器出错	F12 Network 里每条请求的 Status
忽略名单 / .gitignore	一份名单，写在里面的文件 Git 不会上传	.env、node_modules
路由	页面之间的地址或切换规则	/、/platform、/detail
防抖	短时间内连续触发时只认第一次，避免重复执行	连点收藏只写入一条记录
软删除	不真删数据，只加一个已删除标记，查询时跳过	Day 22 余力加练
回归验证	改完之后把原来的功能重新跑一遍，确认没弄坏别的	Day 19 重构后逐个接口重调
外键	指向另一张表某个字段的值，用来把两张表关联起来	favorites.trends_id 指向 trends.id

附录 D | 工具、技术栈和账号路线选择

[返回总目录](#) · [回到 Day 2](#) · [回到 Day 5](#)

工具名称	是否需要安装	用途
WorkBuddy	Day 1 安装	全程使用的桌面 AI 编程助手：读写你的项目文件、运行命令、生成和修改代码、部署上线
Git	Day 1 安装	在你电脑上记录每次改动，是提交到 GitHub 的基础
Node.js	Day 1 安装	运行前端项目、执行 npm 相关命令
GitHub 账号	Day 1 注册	存放项目代码的云端仓库，也是你 28 天的进度记录；班级组织会邀请你加入
浏览器	不需要安装	打开本地页面、公网页面和接口地址；配合 F12 查看报错
浏览器开发者工具 (F12)	不需要安装	查看页面报错、网络请求和接口返回；第 10、20、23、24 天会用到
截图工具	不需要安装	每天交打卡证据，用系统自带截图或常用截图快捷键即可
GitHub Desktop	不需要安装	老师演示用：直观看到每次改了哪些文件、提交历史长什么样。感兴趣的同学可自行安装探索，不装不影响任何任务

CloudBase（腾讯云开发）	Day 15 注册	后端和云端部分：云函数、PostgreSQL 数据库、静态网站托管。属于后端环节，Day 1-14 用不到，现在不需要注册，也不需要实名认证
PostgreSQL	不需要安装	存放作品的数据（表、字段、记录），在 CloudBase 控制台内直接使用
域名	不需要安装	本课程使用 CloudBase 提供的公网地址，不需要购买域名
服务器	不需要安装	使用 CloudBase 的云函数与静态网站托管，不需要租服务器

AI 可以安装项目依赖；CloudBase 的登录、授权、计费 and 删除资源必须由学员确认。换技术栈的学员允许使用等效替代（如 MySQL 代替 PostgreSQL、其他托管平台代替 CloudBase），证据要求不变。

账号体系二选一

路线	适合谁	必须处理	本训练营建议
匿名/演示身份	第一次做全栈项目的人	重复写入、数据清理、隐私说明	默认路线，优先完成核心闭环
注册登录	已有后端经验的人	密码哈希、会话、权限、退出和隐私	选做，不得阻塞部署

默认不要求登录。匿名或演示身份已经足够学习新增、读取、修改和删除。选择注册登录时，必须额外处理密码哈希、会话、权限、退出、隐私和线上环境变量；不要在最后几天临时加入登录，导致核心闭环无法验收。

路线选择提示词

```
代码块

1  请帮我做技术路线选择。
2
3  我的项目叫【项目名称】，现在是第【天数】天，今天的任务是【当天任务】。
4  我纠结的地方：【前端框架 / AI 工具 / 数据库 / 部署平台】
5  候选方案 A：【方案 A】
6  候选方案 B：【方案 B】（如有方案 C 也列出）
7  我的情况：【操作系统、已装工具、编程经验】
8  我的限制：【电脑/时间/账号限制】，费用只接受【免费额度内 / 可接受金额】
9
10 要求：
11 1. 用小白能懂的话解释每个方案是什么、解决什么问题，术语先用一句大白话说明
12 2. 从五个角度逐项比较，每个角度给出"A 更合适 / B 更合适 / 差不多"的结论：
13   - 学习成本
14   - 安装和配置难度
```

- 15 - 费用（说明免费额度和超出后怎么收费）
- 16 - 排错难度（新手好不好搜到答案）
- 17 - 和验收要求的匹配度（公网可访问、数据刷新后仍在、能演示写入/读取/修改/删除）
- 18 3. 用表格输出比较结果
- 19 4. 明确推荐一个方案，用 2-3 句话说原因；两个都能完成验收时，推荐更省事的那个
- 20 5. 如果推荐的不是示范路线（如 MySQL 代替 PostgreSQL、其他平台代替 CloudBase），指出哪
些验收证据要求不变、哪些步骤要相应替换
- 21 6. 最后给我 3 个动手前的自查问题
- 22
- 23 先不要写任何代码；等我回复"确认选择"后再开始。

附录 E | CloudBase PostgreSQL 初始化、备份与持久化

[返回总目录](#) · [回到 Day 16](#) · [回到 Day 27](#)

最小初始化顺序

1. 先让 AI 根据 PRD 输出表、字段、主键、唯一约束和索引，不立即执行。
2. 在 CloudBase PostgreSQL 环境中创建表，并保存可重复执行的初始化/迁移脚本。
3. 运行种子数据，重复执行一次，确认不会产生不可控重复。
4. 通过 API 完成一次写入和读取，再到数据库中核对记录。
5. 导出表结构、初始化脚本和必要数据；备份中不得包含密钥或私人信息。

数据库提示词

代码块

- 1 请读取当前项目的 PRD、TECH_DESIGN 和真实数据库配置，为 CloudBase PostgreSQL 生成可重复执行的初始化、种子和备份方案。
- 2 先列出表、字段、主键、关联、唯一约束和索引，等我确认后再执行。
- 3 连接信息只从环境变量读取，不要输出或要求我粘贴真实值。
- 4 完成后验证：重复初始化、API 读取、API 写入、数据库记录、刷新后的持久化和数据导出。
- 5 任何会覆盖或删除数据的命令都先停下来说明影响和回滚方法（例外：公网发布前、经你逐次确认的初始化脚本含 DROP 属正常操作，见第 16 天；应用上线后，任何未经确认的 DROP/DELETE/UPDATE 一律停下）。

附录 F | 真实数据来源、替换和合规

[返回总目录](#) · [回到 Day 17](#) · [回到 Day 23](#)

允许的来源

优先选择官方 API、公开文档明确允许使用的接口、无需登录的公开 JSON/RSS 或其他合法公开来源。来源政策会变化，发布前要重新检查。

不允许的做法

不绕过验证码、反爬、登录和访问频率限制；不使用他人 Cookie、Token 或 API 密钥；不把受限内容重新分发为自己的数据服务。

代码块

- 1 当前数据源【名称/地址】出现【失效/限流/需要登录】。
- 2 请不要绕过验证码、反爬或登录限制，也不要使用他人 Cookie。
- 3 根据我的字段【字段列表】寻找一个合法公开、无需登录的替代来源，比较稳定性、许可和接入难度。
- 4 先给来源选择和最小改动计划；暂时没有合适来源时，保留并明确标注“示例数据”。

同步提示词模板

代码块

- 1 请为「今日热搜」案例生成数据同步云函数 POST /api/sync。
- 2 运行环境：CloudBase Node.js 18.15，用 Node 内置 fetch，不要引入任何第三方依赖。
- 3
- 4 【数据源（必须原样使用这三个公开接口，不要替换成其他站点或聚合服务）】
- 5 1. 微博 <https://weibo.com/ajax/side/hotSearch>
- 6 2. B站 <https://api.bilibili.com/x/web-interface/search/square?limit=50>
- 7 3. 抖音 https://www.douyin.com/aweme/v1/web/hot/search/list/?device_platform=webapp&aid=6383
- 8
- 9 【请求头（三个平台都必须带，缺一个就会失败，不要省略）】
- 10 - User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
- 11 - Referer: 各平台用自己的站点首页 (<https://weibo.com> / <https://www.bilibili.com> / <https://www.douyin.com/>)
- 12 说明：微博缺 Referer 会返回 403；B站缺桌面 UA 会返回 412；抖音缺 Referer 会返回 200 但列表为空。所以这三个头都不能省。
- 13
- 14 【字段映射（写入 trends 表，字段名用小写英文）】
- 15 - platform: 固定 'weibo' / 'bilibili' / 'douyin'
- 16 - title: 微博 data.realtime[].word；B站 data.trending.list[].keyword；抖音 data.word_list[].word
- 17 - hot: 微博 .num；B站 .heat_score；抖音 .hot_value
- 18 - rank: 微博 .realpos；抖音 .position；B站不返回排名，用数组下标 + 1
- 19 - date: 抓取当日，必须按北京时间（UTC+8）计算
- 20 - fetched_at: 抓取时刻
- 21
- 22 【实现要求】
- 23 1. 每个平台取前 30 条；三个平台并行抓取，单个平台失败不影响其他平台。
- 24 2. date 必须按 UTC+8 生成。云函数默认时区是 UTC，直接用 toISOString() 取日期，会在北京时间 00:00-08:00 之间算成前一天，导致同一天的榜单被写进两批数据。
- 25 3. 写入用「存在则更新、不存在则插入」：唯一索引 (platform, title, date) + INSERT ... ON CONFLICT (platform,title,date) DO UPDATE SET hot=..., rank=..., fetched_at=...。重复执行不产生重复行。

26 4. SQL 必须参数化，禁止字符串拼接。
27 5. 单个平台失败时写清是哪个平台、什么原因，例如「微博接口返回 403，通常是请求头缺少
Referer」。
28 6. 三个平台全部失败时返回 {ok:false, error}，页面保留 seed 并标注「示例数据」。
29
30 【另外给我】
31 - 部署步骤，以及手动触发一次 /api/sync 的方法；
32 - 验证步骤：select 查 trends 应看到三个平台各约 30 行、platform 有
weibo/bilibili/douyin 三种值；再触发第二次 sync，确认总行数不变（证明判重生效）。

附录 G | Skill 创建与验收

[返回总目录](#) · [回到 Day 12](#) · [回到 Day 25](#)

Skill 放在哪、叫什么

一个 Skill 就是一个文件夹，文件夹里放一份 SKILL.md。位置只有两处，本课程用项目级。

- 项目级：项目文件夹/.workbuddy/skills/<技能名>/SKILL.md。跟着项目走，可以提交到 GitHub
- 用户级：~/.workbuddy/skills/<技能名>/SKILL.md。所有项目都能用

项目级优先于用户级，同名时以项目级为准。

技能名的写法（写错会导致调用失败，务必照做）

- 只用小写字母、数字和连字符 -
- 不要用中文、空格、下划线和大写字母
- 文件夹名必须和 SKILL.md 里 name 的值完全一致

写法	是否可用	原因
frontend-guidelines	可用	小写加连字符
verify-project	可用	小写加连字符
FrontendGuidelines	不可用	含大写字母
frontend_guidelines	不可用	含下划线
前端设计规则	不可用	含中文

本课程用到两个 Skill，名字已按上面的规则定好：Day 12 用 frontend-guidelines，Day 25 用 verify-project。

SKILL.md 的格式

文件最开头必须有一段用三个连字符包起来的配置，叫 frontmatter。其中两个字段是必需的。

字段	作用	注意
name	技能名	必须和文件夹名一致
description	一句话说明这个 Skill 干什么	决定 AI 什么时候自动调用它

description 一定要写具体。写成「帮助前端开发」这种笼统描述，Skill 永远不会被触发。

下面是 Day 12 要创建的 Skill 的完整文件内容，可以直接照着写。

```
.workbuddy/skills/frontend-guidelines/SKILL.md
1  ---
2  name: frontend-guidelines
3  description: 本项目的前端页面设计规则，用于检查信息层级、字体、颜色对比、间距和移动端布局
   问题
4  ---
5
6  # 前端设计规则
7
8  ## 页面层级
9  - 每个页面只有一个主标题
10 - 区块之间靠间距分组，不靠加边框
11
12 ## 颜色与字体
13 - 正文 14px，区块标题 18px 以上
14 - 正文与背景的对比度要够，浅灰字不要放在白底上
15
16 ## 卡片与按钮
17 - 卡片内边距 16px，卡片间距 12px
18 - 按钮要有默认、悬停、禁用三种状态
19
20 ## 移动端
21 - 360px 宽度下不能出现横向滚动
22 - 导航在小屏可以折叠
23
24 ## 修改前
25 - 先列出具体的问题，再逐个修复，一次只改一个
26
27 ## 修改后
28 - 桌面宽度和手机宽度各看一遍
29 - 确认交互没有因为改样式而失效
30
```

下面是 Day 25 要创建的那个 Skill 的骨架，检查项按你项目真实踩过的坑替换。

```
.workbuddy/skills/verify-project/SKILL.md

1  ---
2  name: verify-project
3  description: 每次发布前自动检查项目是否达标，覆盖公网访问、接口、密钥、数据库和备份
4  ---
5
6  # 发布前检查
7
8  ## 检查项
9  1. 公网首页可访问，且展示数据库真实数据
10  2. 健康接口 GET /api/health 返回 ok
11  3. 读写接口各调用一次，结果正确
12  4. 代码和 Git 历史里搜不到硬编码密钥
13  5. .gitignore 覆盖 .env，仓库里有 .env.example
14  6. 数据库可连接，核心表存在
15  7. 记录剩余额度和到期时间
16
17  ## 输出格式
18  逐项给出 PASS / FAIL / 未执行，每项附证据。
19  不允许写「看起来正常」「基本没问题」。
20
21  ## 失败规则
22  命令不存在、请求失败或证据缺失时，必须报告 FAIL。
23
```

怎么调用、怎么确认它生效

- 自动触发：你的任务描述和 description 对得上时，AI 自己加载它
- 手动触发：在对话里输入 /技能名，例如 /verify-project
- 确认加载：输入 /skills 命令，能看到 User skills、Project skills、Plugin skills 三组清单。看不到就是没加载成功

最常见的坑是文件夹名和 name 不一致。这种情况下用 /技能名 调用会失败，而且不会给出明确报错，只会说找不到。建完 Skill 后一定回到 /skills 里确认一遍。

检查类 Skill 的内容建议

代码块

- 1 名称：清楚表达它检查什么
- 2 触发方式：什么时候使用
- 3 输入：PRD、package.json、CloudBase 配置和公网地址
- 4 步骤：Git 敏感文件 → 前端构建 → 云函数健康 → 核心读写 → 数据库记录 → 线上地址 → 资源额度

- 5 输出：PASS、FAIL、未执行和实际证据
- 6 失败规则：命令不存在、测试失败或证据缺失时必须报告失败
- 7 验证：正常运行一次，再制造一个可恢复失败，确认不会虚报通过

Skill 验收要点

- Skill 实际调用过至少一次。
- 故意制造失败时没有虚报通过。
- 没有把真实密钥写进 Skill。
- 检查命令来自项目真实脚本。
- 结果可被同伴复核。

附录 H | CloudBase 部署、额度和到期检查

[返回总目录](#) · [回到 Day 26](#) · [回到 Day 27](#)

环节	CloudBase 组件	必须验证
前端	静态网站托管	生产构建成功、公网页面、API 地址
后端	Node.js 云函数 + HTTP 访问服务	健康接口、日志、错误响应
数据库	CloudBase PostgreSQL	建表、真实读写、刷新持久化、导出
配置	环境变量/控制台配置	变量名正确，值不进入代码、Git 或聊天
资源	免费环境/体验额度	剩余额度、到期时间、是否开启按量计费

发布前与发布后清单

- 本地生产构建成功，构建命令来自真实 package.json。
- 云函数健康接口可以调用，HTTP 路由与前端 API 地址一致。
- 公网首页完成一次真实读取和写入，刷新后数据仍存在。
- 云函数日志、Network 请求和数据库记录能对应同一次操作。
- .env、密钥和数据库导出未进入 Git。
- 记录剩余额度、环境到期时间和是否开启按量计费。
- 保存重新发布、回滚和数据导出方法。
- 默认域名和免费环境只在环境有效、额度未耗尽且服务未停用时可访问，不能承诺永久有效。

部署提示词

- 代码请根据当前项目和 CloudBase 官方文档生成发布清单。
- 2 前端部署到 CloudBase 静态网站托管；后端使用 Node.js 云函数；数据库使用 CloudBase PostgreSQL；API 通过 HTTP 访问服务公开。
 - 3 先读取真实 package.json、云函数入口、数据库配置和环境信息，再列出构建命令、部署顺序、环境变量名称、回滚方法和数据导出方法。
 - 4 不要输出密钥，不要开启付费套餐、按量计费或预置并发，不要编造平台命令。
 - 5 发布后验证：公网首页、健康接口、核心读取、核心写入、刷新持久化、云函数日志、数据库记录、剩余额度和到期时间。

结营后作品归属与续命

代码仓库：结营后找管理员把仓库从班级组织转移到你的个人 GitHub 账号（转移步骤问 AI 或老师），作品集里放个人账号链接。

云端环境：CloudBase 免费环境到期后公网地址会失效。到期前用 Day 27 导出的备份 + 仓库代码，在自己的新环境按回滚手册重新部署即可恢复；数据导出方法见[附录 E](#)，部署流程见 Day 15 和 Day 20。

一句话记住：代码在 GitHub 上永久保存；数据靠到期前导出的备份；公网地址是租来的，会到期。

附录 I | 常见问题快速处理

[返回总目录](#) · [回到 Day 23](#) · [回到 Day 26](#)

问题	先检查	不要先做
页面空白	终端、Console、构建命令和第一条报错	大范围重写页面
API 404	方法、路径、HTTP 路由、环境和云函数日志	同时改前后端多个路径
CORS/网络失败	F12 Network 里报错的具体域名、云函数 HTTP 访问服务的跨域配置（做法见第 20 天的 CORS 预置答案）	把 CORS 配置成允许所有来源（*）
数据库连接失败	环境变量名、权限、网络与第一条数据库错误	把连接串写进代码
数据没有保存	写入响应、云函数日志、数据库记录、刷新结果	只相信页面成功提示
部署失败	真实构建脚本、输出目录、日志第一条关键错误	反复点击发布
AI 做了不该做的改动，想撤销	让 AI 执行「恢复到上一次提交，保留历史」（git revert），回退后核对文件状态；更早的问题先让 AI 说明是哪个提交引入的，再由 AI 精准恢复	手动执行 reset --hard、强制推送或删掉文件夹重来
额度/环境到期	控制台用量、到期日、计费状态和备份	未经确认开启按量计费

附录 J | 100 分最终验收与项目映射表

项目	分值	通过证据
提示词理解与改写	20	3 条以上实际改写提示词和复盘
前端页面与交互	15	3 个页面/视图、反馈状态和移动端截图
云函数与 API	15	健康、读取、写入和错误响应
云数据库（PostgreSQL 或等效云数据库）	15	初始化、真实读写、修改/删除和持久化
第三周周成果	10	云端数据服务 v1、检查台和演示证据
Skill 与测试	10	Skill 调用、故意失败、Bug 修复和回归
线上发布（CloudBase 或等效平台）	10	公网地址、线上读写、日志和数据库证据
额度、到期和备份	5	到期时间、计费状态和数据导出说明

我的项目与能力底线对应表

能力底线	我的项目对应功能	证据链接/截图
至少 3 个页面/视图		
至少 1 个有反馈的交互		
至少 1 类数据真实写入并读取		
前端、云函数和数据库完整链路		
一次公网发布（CloudBase 或等效平台）		
刷新后数据仍存在		
额度、到期和备份已记录		
能解释提示词和验证方式		

结营交付清单

- 公网地址
- 代码仓库链接
- 3-5 分钟演示录屏

- [最终验收表和项目映射表](#)
- 一条最有效的改写提示词
- 运行和技术路线说明
- 数据库、部署、额度、到期和备份说明（不含密钥）
- 200 字以内结营总结

附录 K | 浏览器开发者工具（F12）速查

[返回总目录](#) · [回到 Day 10](#)

第 10、20、23、24 天都会用到浏览器开发者工具（Day 10 主要用 Elements 看样式）。这份速查只讲本课程用得到的部分，不求全。

怎么打开：Windows 按 F12 或 Ctrl+Shift+I，Mac 按 Cmd+Option+I；也可以在页面上右键选「检查」。打开后是一大块面板，顶部有一排标签页，本课程常用其中三个。

五个常用面板：

面板	是什么	什么时候看它
Console（控制台）	显示页面的报错和警告，红色 = 出错	前端出问题第一个看这里：白屏、按钮点了没反应、数据不显示
Network（网络）	显示页面发出的每个请求：地址、状态码、返回内容	接口调不通、显示的数据不对、怀疑跨域时
Elements（元素）	显示页面的结构和样式	样式乱了、找不到某个元素时（配合第 10 天的截图定位）
Application（应用）	查看本地存储和 Cookie	本课程偶尔用到，知道有它就行
Sources（源代码）	源码级调试	本课程基本用不到，知道有它就行

三个最常用场景，照着做：

- 接口调不通怎么查：**按 F12 → 点 Network 标签 → 刷新页面 → 在列表里找到变红的请求 → 点开看 Status（状态码，200 = 成功，4xx/5xx = 出错）和 Response（返回内容）→ 把状态码和返回内容一并发给 AI。请求地址是不是公网地址也在这里看。
- 页面白屏怎么查：**按 F12 → 点 Console 标签 → 找第一条红色报错 → 复制报错原文发给 AI。发文字比发截图好，文字能直接定位问题。
- 样式乱了怎么查：**右键出问题的元素 → 选「检查」→ 在 Elements 里看被选中的元素和右侧样式面板，逐条看哪条样式被划掉或没生效。

安全提醒：截图发群之前，检查 Network 面板的请求内容里有没有密钥、Token、数据库连接串——有就先打码或删除再发。

附录 L | 项目文件命名规范

[返回总目录](#) · [回到 Day 1](#)

全篇出现的文件名大小写并不统一，容易让人怀疑自己是不是写错了。按下面这张表来就行。

类型	文件名	出现位置
项目规则文件	AGENTS.md	Day 1 创建，Day 6 追加
需求研究	research.md	Day 3
需求文档	PRD.md	Day 4
技术设计	TECH_DESIGN.md	Day 5
接口契约	api-contract.md	Day 15
建表脚本	db/schema.sql、db/seed.sql	Day 16
环境变量样例	.env.example	Day 23
忽略名单	.gitignore	Day 2、23
Skill 定义	.workbuddy/skills/<技能名>/SKILL.md	Day 12、25

一句话记住这个约定：**AGENTS.md、PRD.md、TECH_DESIGN.md** 这三类早期规范文档，以及 **.gitignore、.env.example** 这类配置文件用大写；其余文件一律小写、用连字符分隔（如 **research.md、api-contract.md**）。拿不准就以[附录 L](#) 这张表为准，不要自己推理。

另外两条容易踩的坑：Skill 的文件夹名必须和 SKILL.md 里的 name 完全一致（见[附录 G](#)）；忽略名单的文件名就是 `.gitignore`，注意前面有一个点，不要写成「忽略文件」。

附录 M | CloudBase 注册、控制台与使用指导手册

[返回总目录](#) · [回到 Day 15](#) · [回到 Day 20](#) · [回到 Day 27](#)

本手册供未看直播、进度较快或需要回查的学员按文档自学；培训课与手册内容一致，以文档为准。CloudBase 属于后端环节，从第 15 天才开始用，Day 1-14 不需要注册、不需要实名认证、不需要做任何准备。

一、注册入口与流程（约 5-10 分钟）

1. 打开腾讯云 CloudBase（云开发）官网，用微信或手机号登录；
2. 按提示完成注册和实名认证（个人认证即可，免费）；

3. 创建环境时只选免费体验环境。控制台若引导你开通按量计费或付费套餐，先停下来——本课程不
开通（见下方安全红线）。

二、环境 ID、剩余额度、到期时间（Day 15 当天就记下）

- 1. 环境 ID：控制台环境设置里的一串字符，记到项目笔记；
- 2. 剩余额度：控制台的用量/费用页查看；
- 3. 到期时间：免费体验环境的到期日期，和额度一起记下；
- 4. 具体入口以控制台实际界面为准；找不到就让 AI 根据你截图的页面指路，也可以对照[附录 H](#) 的资源检查清单。

三、控制台地图：每个环节在哪天用

控制台环节	用到的天数
云函数	Day 15（第一个 /api/health）、Day 17-19（读接口/写接口/分层重构）、Day 22（修改/删除）
HTTP 访问服务	Day 15（公网地址）、Day 20（跨域 CORS 配置就在这里）
静态网站托管	Day 15（前端 mock 版上线）、Day 20（真实数据版重新上传）、Day 26（回滚演练）
PostgreSQL 数据库	Day 16（建表/种子）、Day 17-22（验证读写）、Day 27（数据导出）
用量与费用	Day 15（记录额度/到期）、Day 26/27（资源体检）

四、安全红线

不开启按量计费、预置并发或任何付费套餐（与[附录 B](#) 的 AI 操作边界一致）；数据库连接串和密钥只放环境变量，不进代码、Git 和聊天；到期前完成数据导出（方法见[附录 E](#)）。

五、常见问题

注册、开通、额度类问题先看[附录 I](#) 的「额度/环境到期」一行；部署类问题看 Day 15 的「容易吃亏的事」和 Day 20 的 CORS 预置答案。